Données et stratégie avec Python

MGMT 60XXX · Enjeux locaux et mondiaux · HEC Montréal

Thierry Warin, PhD

6 novembre 2026

Table des matières

Avant-propos **1**
Comment lire ce livre 1
L'IA générative 2

I Python pour la stratégie **3**

1 Installer la plateforme **5**
1.1 D'une manchette à une question 5
1.2 La plateforme : quatre pièces 5
1.3 Python en quelques lignes 6
1.4 Le premier tableau de données 7
1.5 Une première réponse, et ses limites 9
1.6 L'IA générative comme partenaire d'étude 10

2 Le DataFrame, la table de l'analyste **11**
2.1 Une question qui n'a pas une réponse 11
2.2 Le fichier 11
2.3 L'anatomie d'un DataFrame 13
2.4 Regarder avant de calculer 14
2.5 Sélectionner et filtrer 14
2.6 Quatre mesures, quatre classements 15
2.7 Quand un ratio ment 16
2.8 Comparer à l'intérieur des secteurs 16
2.9 Une figure : taille contre efficacité 17
2.10 Les valeurs manquantes 17
2.11 Une note sur pandas 3 19
2.12 À retenir 19

3 Importer des données **21**
3.1 Une question, plusieurs sources 21
3.2 Le fichier CSV, et ce qu'il ne dit pas 21
3.3 Lire en ligne, une fois 23
3.4 Les paquets qui parlent aux sources 24
3.5 Les API 27
3.6 Les bases de données 28
3.7 Sauvegarder et documenter 30
3.8 À retenir 30

4 Manipuler les données : nettoyer, joindre, remodeler, agréger **33**
4.1 Ce que l'on ne voit pas dans un article 33

4.2 Nettoyer . . . 33
4.3 Joindre . . . 35
4.4 Remodeler . . . 36
4.5 Agréger, et comparer chaque entreprise à son groupe . . . 37
4.6 Le temps, entité par entité . . . 40
4.7 Les fusions-acquisitions créent-elles de la valeur ? Un premier regard 41
4.8 À retenir . . . 43

5 Représenter les données 45
5.1 Une manchette, plusieurs vérités . . . 45
5.2 Pourquoi regarder plutôt que résumer . . . 45
5.3 Ce que l'œil lit bien . . . 46
5.4 L'anatomie d'une figure . . . 47
5.5 Choisir le graphique selon la question . . . 49
5.6 seaborn : la statistique en une ligne . . . 49
5.7 plotly : explorer avant de conclure . . . 50
5.8 Les graphiques qui trompent sans mentir . . . 52
5.9 À retenir . . . 52

6 Statistiques de base : décrire, comparer, relier 53
6.1 Ce que l'on voit, et ce que l'on peut affirmer . . . 53
6.2 Décrire . . . 53
6.3 De l'échantillon à la population . . . 55
6.4 Comparer . . . 56
6.5 Relier . . . 59
6.6 Corrélation n'est pas causalité . . . 61
6.7 Ce que l'on peut écrire . . . 62

II Articles et études de cas 65

7 Des outils aux questions : les séances 7 à 13 67
7.1 Lire un cas avec les yeux d'un analyste . . . 68
7.2 Du cas au projet . . . 68

Annexes 69

A Installation pas à pas 69
A.1 Python . . . 69
A.2 VS Code . . . 69
A.3 Le dossier du cours et l'environnement virtuel . . . 69
A.4 Vérifier . . . 69
A.5 Dépannage . . . 70

B Sources de données **71**
B.1 `entreprises.csv` 72
B.2 `pays.csv` 73
B.3 Autres sources utiles pour le projet 73

C Aide-mémoire Python **75**
C.1 Lire et écrire 75
C.2 Regarder 75
C.3 Sélectionner et créer 75
C.4 Transformer 76
C.5 Représenter 76
C.6 Tester 77

Avant-propos

Chaque semaine, la presse d'affaires publie des phrases qui ressemblent à des faits. *Tesla est bonne pour l'environnement. L'Amérique crée plus de valeur que tout autre pays. Les entreprises qui investissent en recherche sont plus rentables. Les fusions détruisent de la valeur.* Chacune contient une part de vérité, et aucune n'est une question à laquelle des données peuvent répondre telle quelle. Avant de les accepter ou de les rejeter, il faut les réécrire : bonne *par rapport à quoi*, plus de valeur *selon quelle mesure*, plus rentables *en moyenne, dans quel secteur, à quel horizon* ?

Ce livre accompagne le cours MGMT 60XXX, *Données et stratégie : enjeux locaux et mondiaux*, à HEC Montréal. Il couvre les six séances qui donnent les outils : installer une plateforme de travail, comprendre le tableau de données, importer des données depuis un fichier, une adresse web, un paquet, une API ou une base de données, les transformer, les représenter et les tester. Les séances suivantes, données par mes collègues, mobilisent ces outils au service de questions de recherche en stratégie, à partir d'articles et d'études de cas.

Le parti pris du livre tient en trois principes.

Des données réelles, dès la première page. Les entreprises du livre sont Tesla, Ford, Microsoft, Pfizer, Walmart ; leurs chiffres viennent des états financiers déposés à la SEC. Les pays viennent de la Banque mondiale. Ces données ont des trous, des ruptures, des valeurs extrêmes : on les garde, parce que c'est ce que vous rencontrerez. Un jeu de données pédagogique trop propre enseigne une compétence qui n'existe pas.

Regarder avant de théoriser — et le dire. En stratégie, la recherche guidée exclusivement par une théorie préalable passe parfois à côté de ce que les données montrent (D. Miller 2007). Observer, décrire et comparer avec soin n'est pas une étape préliminaire : c'est souvent là que se trouve le résultat, y compris lorsqu'il est nul (C. C. Miller et Bamberger 2016). La condition est la transparence : dire ce que l'on a cherché, et comment.

Le code est la méthode. Chaque tableau et chaque graphique du livre est produit par du code Python exécuté au moment de la construction du livre. Si un chiffre change, c'est que les données ont changé. Le projet final du cours demande la même chose : un rapport, et le fichier de code qui le reproduit.

Comment lire ce livre

Chaque chapitre correspond à une séance et suit la même structure qu'elle : une question de stratégie, les concepts, le code, et ce que les données ne permettent pas de conclure. Le code s'exécute depuis la racine du dépôt du cours : `pd.read_csv("data/entreprises.csv")` suppose que le terminal est ouvert dans le dossier qui contient `data/`.

Les annexes rassemblent l'installation pas à pas, la description des sources de données et un aide-mémoire des fonctions Python utilisées.

L'IA générative

Les assistants d'IA générative écrivent du Python correct la plupart du temps — et du Python faux avec la même assurance le reste du temps. Ils sont utiles pour expliquer un message d'erreur, commenter un bloc de code ou vous interroger sur un concept. Ils ne remplacent pas la vérification. La règle du cours est simple : tout usage dans un travail remis est déclaré, avec ce que vous avez vérifié et comment.

Première partie

Python pour la stratégie

Chapitre 1

Installer la plateforme

1.1 D'une manchette à une question

« Tesla est vraiment bonne pour l'environnement. » La phrase circule, elle est plausible, et elle ne peut être ni confirmée ni infirmée telle quelle. Pour qu'une donnée puisse lui répondre, il faut d'abord préciser trois choses.
Par rapport à quoi ? Un véhicule électrique se compare à un véhicule thermique de même catégorie, à la moyenne du parc, au transport en commun — et la réponse change selon le comparateur. *Selon quelle mesure ?* Les émissions à l'usage, à la fabrication (la batterie pèse lourd) ou sur l'ensemble du cycle de vie ne racontent pas la même histoire. *Pour qui, et où ?* Une voiture électrique rechargée au Québec, où l'électricité est presque entièrement hydraulique, n'a pas le même bilan qu'en Pologne, où le charbon domine encore.
Ce travail de réécriture est la première compétence du cours. Il ne demande aucun code. Mais dès qu'une question est précise, il faut des données, et un outil pour les lire. Ce chapitre installe cet outil.

1.2 La plateforme : quatre pièces

Travailler avec des données en Python mobilise quatre éléments qu'il est utile de distinguer, parce que la plupart des problèmes d'installation viennent de leur confusion.

Pièce	Ce que c'est
Python	Le langage, et l'*interpréteur* qui exécute le code
Les paquets	Du code écrit par d'autres (`pandas`, `matplotlib`…), que l'on importe
L'environnement virtuel	Une installation de Python propre à un projet, avec ses paquets et leurs versions
L'éditeur (VS Code)	Le logiciel où l'on écrit et exécute le code

L'environnement virtuel est la pièce la moins intuitive. Sans lui, tous les projets d'un ordinateur partagent les mêmes paquets ; le jour où deux projets exigent des versions incompatibles, l'un des deux cesse de fonctionner. Avec lui, chaque projet a son dossier `.venv`, et la liste de ses paquets tient dans un fichier, `requirements.txt`, que n'importe qui peut réinstaller à l'identique :

```
python3 -m venv .venv                          # Windows : py -m venv .venv
source .venv/bin/activate                      # Windows : .venv\Scripts\activate
python -m pip install -r requirements.txt
```

> **Le piège des certificats sous macOS.** L'installateur de python.org n'installe pas les certificats racines utilisés par le module `ssl`. Tant que le script `Install Certificates.command` (dans `Applications/Python 3.x`) n'a pas été exécuté, une instruction comme `pd.read_csv("https://…")` échoue avec `CERTIFICATE_VERIFY_FAILED`, alors même que le navigateur accède sans difficulté à la même adresse. Le script `verifier_environnement.py` de la séance 1 teste précisément ce cas.

1.3 Python en quelques lignes

Python se lit presque comme de l'anglais. Le signe = range une valeur dans un nom :

```
chiffre_affaires = 96_773   # Tesla, 2023, millions de $ US
resultat_net = 14_974
marge_nette = resultat_net / chiffre_affaires
print(f"Marge nette de Tesla en 2023 : {marge_nette:.1%}")
```

```
Marge nette de Tesla en 2023 : 15.5%
```

Chaque valeur a un **type**. Les plus courants sont l'entier (`int`), le nombre à virgule (`float`), le texte (`str`) et le booléen (`bool`). La confusion entre un nombre et du texte qui ressemble à un nombre est l'erreur la plus fréquente lors de l'importation de données :

```
print(2023 + 1, "2023" + "1")
```

```
2024 20231
```

Deux structures suffisent pour commencer. La **liste** range des valeurs dans un ordre ; le **dictionnaire** associe des clés à des valeurs :

```
constructeurs = ["Tesla", "Ford", "General Motors"]
ventes_2023 = {"Tesla": 96_773, "Ford": 176_191, "General Motors": 157_658}
for nom in constructeurs:
    print(f"{nom:15s} {ventes_2023[nom]:>9,} M$")
```

```
Tesla              96,773 M$
Ford              176,191 M$
General Motors    157,658 M$
```

Enfin, une **fonction** donne un nom à un calcul que l'on répétera. Si la définition change, on la corrige à un seul endroit :

```
def marge(resultat, ventes):
    """Part de chaque dollar de ventes qui reste en profit."""
    return resultat / ventes

print(f"Ford : {marge(4_347, 176_191):.1%}")
```

```
Ford : 2.5%
```

1.4 Le premier tableau de données

Un dictionnaire de listes a déjà la forme d'un tableau : chaque clé est une colonne. Le paquet **pandas** (McKinney 2010) transforme cette structure en *DataFrame*, l'objet central de tout le cours :

```
df = pd.DataFrame({
    "entreprise": ["Tesla", "Ford", "General Motors"],
    "ventes":     [96_773, 176_191, 157_658],
    "resultat":   [14_974, 4_347, 9_840],
})
df["marge"] = df["resultat"] / df["ventes"]
df
```

	entreprise	ventes	resultat	marge
0	Tesla	96773	14974	0.154733
1	Ford	176191	4347	0.024672
2	General Motors	157658	9840	0.062414

Une opération écrite sur une colonne s'applique à toutes les lignes à la fois : c'est ce qui rend pandas rapide à écrire et rapide à exécuter.
Les données du cours sont plus riches. Le fichier `data/entreprises.csv` réunit 32 grandes entreprises cotées aux États-Unis, de 2010 à 2024, à partir des états financiers qu'elles déposent auprès de la SEC :

```
entreprises = pd.read_csv("data/entreprises.csv")
print(entreprises.shape)
entreprises.head()
```

```
(480, 12)
```

	ticker	entreprise	cik	secteur	pays_siege	annee	chiffre_affaires	resultat_net	actifs_totaux	capitaux_propres	rd	investissements
0	TSLA	Tesla	1318605	Automobile	USA	2010	116.7	-154.3	386.1	207.0	93.0	40.2
1	TSLA	Tesla	1318605	Automobile	USA	2011	204.2	-254.4	713.4	224.0	209.0	184.2
2	TSLA	Tesla	1318605	Automobile	USA	2012	413.3	-396.2	1114.2	124.7	274.0	239.2
3	TSLA	Tesla	1318605	Automobile	USA	2013	2013.5	-74.0	2416.9	667.1	232.0	264.2
4	TSLA	Tesla	1318605	Automobile	USA	2014	3198.4	-294.0	5830.7	911.7	464.7	969.9

```
entreprises.isna().mean().round(2).sort_values(ascending=False).head(4)
```

```
rd                  0.28
investissements     0.25
resultat_net        0.02
chiffre_affaires    0.01
dtype: float64
```

La dépense de R-D manque pour plus d'un quart des lignes. Ce n'est pas un défaut de téléchargement : Walmart, Costco ou McDonald's ne publient pas de poste « recherche et développement », et Amazon range la sienne sous un autre intitulé. Une valeur manquante est souvent une information sur l'entreprise ; la remplacer par zéro serait une erreur.

1.5 Une première réponse, et ses limites

Quel constructeur automobile était le plus rentable en 2023 ?

```
auto = entreprises[entreprises["secteur"] == "Automobile"].copy()
auto["marge"] = auto["resultat_net"] / auto["chiffre_affaires"]
(auto[auto["annee"] == 2023]
     [["entreprise", "chiffre_affaires", "resultat_net", "marge"]]
     .sort_values("marge", ascending=False))
```

	entreprise	chiffre_affaires	resultat_net	marge
13	Tesla	96773.0	14974.0	0.154733
43	General Motors	157658.0	9840.0	0.062414
28	Ford	176191.0	4347.0	0.024672

Et depuis quand ?

```
from plotstyle import setup
plt = setup()

fig, ax = plt.subplots()
for nom, groupe in auto.groupby("entreprise"):
    ax.plot(groupe["annee"], groupe["marge"] * 100, marker="o", label=nom)
ax.axhline(0, color="grey", linewidth=0.8)
ax.set_ylim(-30, 20)
ax.set_ylabel("Marge nette (%)")
ax.legend(frameon=False)
plt.show()
```

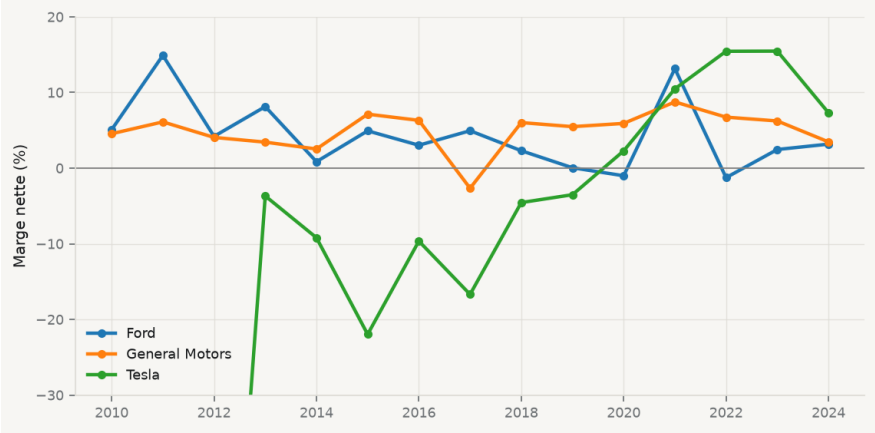


Figure 1.1. Marge nette des trois constructeurs, 2010–2024. Source : SEC EDGAR.

Tesla perd de l'argent chaque année jusqu'en 2019 — en 2010, ses pertes représentent 132% de ses ventes, hors du cadre de la figure Figure 1.1. Elle devient rentable en 2020 et dépasse Ford et GM à partir de 2022.
Ces chiffres répondent à une question de **rentabilité**. Ils ne disent rien de l'impact environnemental de Tesla : il faudrait pour cela des données d'émissions, de mix électrique et de ventes par motorisation, qui ne figurent pas dans des états financiers. Les importer est l'objet du chapitre 3. Le réflexe à acquérir dès maintenant est de toujours écrire, à côté d'un résultat, la question à laquelle il répond — et celle à laquelle il ne répond pas.

1.6 L'IA générative comme partenaire d'étude

Un assistant d'IA générative est utile à trois moments : pour **expliquer** un message d'erreur, pour **commenter** un bloc de code ligne par ligne, et pour vous **interroger** sur un concept. Il est dangereux à un quatrième : lorsqu'il produit un résultat que vous ne savez pas vérifier. Les modèles écrivent souvent du code destiné à d'anciennes versions des paquets — `DataFrame.append`, par exemple, n'existe plus depuis pandas 2.0. Exécuter, lire l'erreur, corriger : la boucle est la même que pour votre propre code.

> **À retenir.** Une question précise avant les données ; un environnement virtuel par projet ; un script qui s'exécute de haut en bas ; et, à côté de chaque résultat, ce qu'il ne permet pas de conclure.

Chapitre 2

Le DataFrame, la table de l'analyste

2.1 Une question qui n'a pas une réponse

« Quelle entreprise crée le plus de valeur ? » La question paraît simple, et un gestionnaire pressé y répondra par le chiffre le plus visible : le profit. Mais le profit mesure la **taille** d'une réussite, pas son **efficacité**. Une entreprise qui gagne deux fois plus qu'une autre avec cinq fois plus d'actifs n'a pas mieux utilisé ce qu'on lui a confié. La stratégie s'intéresse à l'avantage concurrentiel, c'est-à-dire à la capacité de faire *mieux que les concurrents avec les ressources dont on dispose* — et cela exige des ratios.
Ce chapitre en retient quatre :

Mesure	Formule	Elle répond à…
Résultat net	profit après impôts	Qui gagne le plus d'argent, en dollars ?
Marge nette	résultat net / chiffre d'affaires	Que reste-t-il de chaque dollar vendu ?
ROA	résultat net / actifs totaux	Que rapporte chaque dollar d'actifs ?
ROE	résultat net / capitaux propres	Que rapporte chaque dollar des actionnaires ?

Aucune n'est « la » création de valeur — la finance la définit par rapport au coût du capital, ce que nous ne mesurons pas ici. L'objectif est plus modeste et plus utile : apprendre à calculer ces mesures sur des données réelles, et voir **pourquoi elles ne s'accordent pas**. L'outil qui le permet est le DataFrame de pandas, conçu précisément pour ce genre de tableau (McKinney 2010, 2022).

2.2 Le fichier

```
entreprises = pd.read_csv("data/entreprises.csv")
entreprises.head()
```

	ticker	entreprise	cik	secteur	pays_siege	annee	chiffre_affaires	resultat_net	actifs_totaux	capitaux_propres	rd	investissements
0	TSLA	Tesla	1318605	Automobile	USA	2010	116.7	-154.3	386.1	207.0	93.0	40.2
1	TSLA	Tesla	1318605	Automobile	USA	2011	204.2	-254.4	713.4	224.0	209.0	184.2
2	TSLA	Tesla	1318605	Automobile	USA	2012	413.3	-396.2	1114.2	124.7	274.0	239.2
3	TSLA	Tesla	1318605	Automobile	USA	2013	2013.5	-74.0	2416.9	667.1	232.0	264.2
4	TSLA	Tesla	1318605	Automobile	USA	2014	3198.4	-294.0	5830.7	911.7	464.7	969.9

Le fichier compte 480 lignes et 12 colonnes : 32 entreprises cotées aux États-Unis, chacune observée sur quinze exercices (2010-2024). Une ligne n'est donc pas une entreprise, mais **une entreprise une année donnée**. Les montants sont en millions de dollars US, tirés des états financiers déposés à la SEC. Savoir ce que représente une ligne est la première chose à établir devant un tableau ; presque toutes les erreurs de la suite viennent d'une réponse floue à cette question.

2.3 L'anatomie d'un DataFrame

Un DataFrame est un ensemble de **Series** — des colonnes — qui partagent un même **index**, l'étiquette de chaque ligne. Chaque Series a un **type**.

```
entreprises.dtypes
```

```
ticker                 str
entreprise             str
cik                  int64
secteur                str
pays_siege             str
annee                int64
chiffre_affaires   float64
resultat_net       float64
actifs_totaux      float64
capitaux_propres   float64
rd                 float64
investissements    float64
dtype: object
```

Les montants sont des `float64`, même lorsqu'ils sont ronds : il leur manque des valeurs, et la valeur manquante de pandas, `NaN`, est un nombre à virgule. Le texte (`entreprise`, `secteur`) est de type chaîne. Un type inattendu — un montant lu comme du texte, par exemple — est le symptôme le plus fréquent d'une importation ratée ; la séance 3 y reviendra.
L'index par défaut numérote les lignes. On peut le remplacer par une colonne qui a du sens, ce qui permet ensuite de désigner une ligne par son nom :

```
a23_idx = entreprises[entreprises["annee"] == 2023].set_index("ticker")
a23_idx.loc[["TSLA", "F", "GM"], ["entreprise", "chiffre_affaires", "resulta
```

ticker	entreprise	chiffre_affaires	resultat_net
TSLA	Tesla	96773.0	14974.0
F	Ford	176191.0	4347.0

ticker	entreprise	chiffre_affaires	resultat_net
GM	General Motors	157658.0	9840.0

2.4 Regarder avant de calculer

Trois commandes donnent l'essentiel : `shape` (la taille), `info()` (les types et le nombre de valeurs présentes par colonne) et `describe()` (le résumé numérique).

```
entreprises[["chiffre_affaires", "resultat_net", "actifs_totaux", "rd"]].descr
```

	chiffre_affaires	resultat_net	actifs_totaux	rd
count	477.0	469.0	475.0	345.0
mean	103363.0	11698.0	126668.0	6412.0
std	118947.0	16369.0	105906.0	7898.0
min	117.0	-22440.0	386.0	62.0
25%	34204.0	3273.0	44370.0	1150.0
50%	63627.0	6513.0	92918.0	3298.0
75%	116609.0	14136.0	180443.0	8494.0
max	674538.0	100118.0	624894.0	49326.0

Deux lectures s'imposent. Le minimum de `resultat_net` est négatif : des entreprises perdent de l'argent certaines années. Et la moyenne dépasse largement la médiane (la ligne `50%`) : quelques géants tirent la moyenne vers le haut. Sur des données d'entreprises, la médiane est souvent le meilleur résumé du « cas typique ».

2.5 Sélectionner et filtrer

Les crochets simples avec un nom renvoient une Series ; les crochets doubles avec une liste de noms renvoient un DataFrame. Pour les lignes, `.loc` sélectionne par étiquette et `.iloc` par position. Mais l'outil le plus utilisé est le **masque booléen** : une condition sur une colonne produit une Series de vrais et de faux, qui garde les lignes vraies.

```
entreprises[
    (entreprises["annee"] == 2023)
    & entreprises["secteur"].isin(["Automobile", "Énergie"])
][["entreprise", "secteur", "chiffre_affaires", "resultat_net"]]
```

	entreprise	secteur	chiffre_affaires	resultat_net
13	Tesla	Automobile	96773.0	14974.0
28	Ford	Automobile	176191.0	4347.0
43	General Motors	Automobile	157658.0	9840.0
283	ExxonMobil	Énergie	344582.0	36010.0
298	Chevron	Énergie	196913.0	21400.0
313	ConocoPhillips	Énergie	48522.0	10957.0

Les conditions se combinent avec `&` (et), `|` (ou) et `~` (non), chacune entre parenthèses. La méthode `query` exprime la même chose sous forme de phrase :

```
entreprises.query("annee == 2023 and resultat_net < 0")[["entreprise", "resu
```

	entreprise	resultat_net
418	Boeing	-2222.0
448	3M	-6995.0

2.6 Quatre mesures, quatre classements

Créer une colonne revient à écrire une formule sur des colonnes entières ; pandas l'applique à toutes les lignes d'un coup.

```
df = entreprises.copy()
df["marge_nette"] = df["resultat_net"] / df["chiffre_affaires"]
df["roa"] = df["resultat_net"] / df["actifs_totaux"]
df["roe"] = df["resultat_net"] / df["capitaux_propres"]
df["intensite_rd"] = df["rd"] / df["chiffre_affaires"]
a23 = df[df["annee"] == 2023]
```

Classons maintenant les entreprises de 2023 selon chaque mesure.

```
classements = pd.DataFrame({
    m: a23.nlargest(5, m)["entreprise"].tolist()
    for m in ["resultat_net", "marge_nette", "roa", "roe"]
}, index=range(1, 6))
classements
```

Table 2.7. Les cinq premières entreprises de 2023 selon quatre mesures.

	resultat_net	marge_nette	roa	roe
1	Apple	Nvidia	Nvidia	Apple
2	Alphabet	Johnson & Johnson	Apple	Nvidia
3	Microsoft	Microsoft	Johnson & Johnson	Caterpillar
4	Meta Platforms	McDonald's	Alphabet	Johnson & Johnson
5	ExxonMobil	Meta Platforms	Meta Platforms	PepsiCo

Combien d'entreprises figurent dans les quatre classements à la fois ? 0. Les géants de la technologie dominent en dollars ; d'autres entreprises, plus petites ou plus légères en actifs, dominent en efficacité. La question « qui crée le plus de valeur ? » n'a de réponse qu'une fois la mesure choisie — et le choix de la mesure est une décision stratégique, qu'il faut justifier.

2.7 Quand un ratio ment

Un ratio n'a de sens que si son dénominateur en a un. Les capitaux propres peuvent devenir négatifs, lorsqu'une entreprise a racheté massivement ses actions ou accumulé des pertes.

```
neg[["entreprise", "resultat_net", "capitaux_propres", "roe"]].round(3)
```

	entreprise	resultat_net	capitaux_propres	roe
388	McDonald's	8469.0	-4707.0	-1.799
403	Starbucks	4124.5	-8616.0	-0.479
418	Boeing	-2222.0	-17233.0	0.129

3 entreprises sont dans ce cas en 2023. Une entreprise rentable y affiche un ROE négatif ; Boeing, qui perd de l'argent, affiche un ROE positif, parce que deux nombres négatifs donnent un quotient positif. Aucun de ces chiffres ne mesure quoi que ce soit. La bonne réponse n'est pas de les « corriger », mais de les exclure du classement par ROE, et de le dire.

2.8 Comparer à l'intérieur des secteurs

Un taux de marge n'a pas la même signification d'un secteur à l'autre : un détaillant vit de volumes et de marges minces, un éditeur de logiciels de marges épaisses. `groupby` découpe le tableau par secteur, applique un résumé à chaque morceau et combine les résultats.

```
(a23.groupby("secteur")
    .agg(n=("ticker", "count"),
```

```
        marge_mediane=("marge_nette", "median"),
        roa_median=("roa", "median"),
        intensite_rd=("intensite_rd", "mean"))
    .sort_values("roa_median", ascending=False)
    .round(3))
```

Table 2.9. Résumé par secteur, exercice 2023.

secteur	n	marge_mediane	roa_median	intensite_rd
Technologie	7	0.253	0.170	0.170
Restauration	2	0.223	0.146	NaN
Biens de consommation	4	0.145	0.116	0.017
Médias et divertissement	1	0.160	0.111	0.079
Énergie	3	0.109	0.096	0.002
Commerce de détail	4	0.032	0.068	NaN
Pharmaceutique	4	0.098	0.046	0.292
Industrie	4	0.063	0.038	0.036
Automobile	3	0.062	0.036	0.044

La colonne `n` n'est pas décorative : une médiane calculée sur trois entreprises, ou sur une seule, ne décrit pas un secteur. Un résumé sans son effectif ne se lit pas.

2.9 Une figure : taille contre efficacité

```
from plotstyle import setup, CL
plt = setup()

fig, ax = plt.subplots(figsize=(7.2, 4.2))
ax.scatter(a23["chiffre_affaires"] / 1000, a23["roa"] * 100, color=CL.accent
for _, r in a23.nlargest(4, "roa").iterrows():
    ax.annotate(r["entreprise"], (r["chiffre_affaires"] / 1000, r["roa"] * 1
                xytext=(5, 3), textcoords="offset points", fontsize=8)
ax.axhline(0, color=CL.muted, lw=0.8)
ax.set_xlabel("Chiffre d'affaires (milliards $ US)")
ax.set_ylabel("ROA (%)")
plt.show()
```

2.10 Les valeurs manquantes

La colonne `rd` contient 135 valeurs manquantes. Elles ne sont pas réparties au hasard :

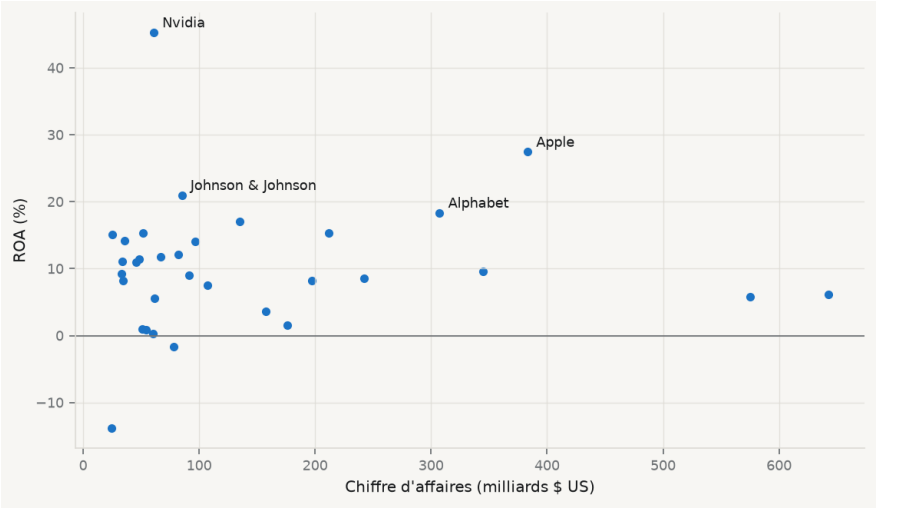


Figure 2.1. Chiffre d'affaires et ROA en 2023. La taille et l'efficacité ne vont pas ensemble.

```
(entreprises.groupby("secteur")["rd"]
    .apply(lambda s: s.isna().mean())
    .sort_values(ascending=False)
    .round(2))
```

```
secteur
Commerce de détail          1.00
Restauration                1.00
Biens de consommation       0.52
Automobile                  0.24
Technologie                 0.03
Industrie                   0.00
Médias et divertissement    0.00
Pharmaceutique              0.00
Énergie                     0.00
Name: rd, dtype: float64
```

Les détaillants et les restaurants ne déclarent pas de ligne de recherche et développement dans leurs états financiers. Ce n'est pas une R-D nulle ; c'est une R-D **non mesurée**. Remplacer ces valeurs par zéro affirmerait quelque chose que la source ne dit pas. Par défaut, pandas ignore les valeurs manquantes dans `mean`, `median` ou `sum` — ce qui est commode, mais signifie que la moyenne porte sur un sous-ensemble, qu'il faut nommer.

2.11 Une note sur pandas 3

Depuis pandas 3.0, toute sélection se comporte comme une copie (*copy-on-write*) : modifier un sous-tableau ne modifie jamais le tableau d'origine. Pour changer l'original, on écrit dans l'original avec `.loc[lignes, colonne] = valeur`. Le vieil avertissement `SettingWithCopyWarning`, que l'on trouve encore dans beaucoup de réponses en ligne et de code généré par l'IA, a disparu.

2.12 À retenir

— Une ligne de `entreprises.csv` est une entreprise **une année** ; établissez toujours ce que représente une ligne.
— `[]` donne une Series, `[[ ]]` un DataFrame ; `.loc` sélectionne par étiquette, `.iloc` par position ; les masques se combinent avec `&`, `|`, `~`.
— La taille et la performance donnent des classements différents ; le choix de la mesure se justifie.
— Regardez le dénominateur avant d'interpréter un ratio.
— Comparez d'abord à l'intérieur des secteurs, et donnez toujours l'effectif d'un résumé.
— Une valeur manquante est une information sur la source.

Chapitre 3

Importer des données

3.1 Une question, plusieurs sources

À la séance 1, la manchette « Tesla est vraiment bonne pour l'environnement » est devenue trois questions : bonne par rapport à quoi, selon quelle mesure, et pour qui, où. Aucune des trois ne se tranche avec un seul fichier. La taille et la rentabilité de Tesla viennent de ses états financiers ; le contenu en carbone de l'électricité qui recharge ses voitures vient de statistiques nationales ; ce que les investisseurs pensent de son avenir se lit dans les cours boursiers. Chaque source est produite par une organisation différente, et servie d'une façon différente.

C'est la condition ordinaire de la recherche en gestion internationale. Bosma et van Witteloostuijn montrent que l'essor de l'apprentissage automatique dans le domaine tient moins à de nouveaux algorithmes qu'à l'arrivée de nouvelles sources de données — textes, données administratives, données en ligne — qu'il faut d'abord savoir faire entrer dans un tableau (Bosma et Witteloostuijn 2024). Ce chapitre traite de cette étape : faire venir les données jusqu'à soi, sans les abîmer en chemin, et en sachant d'où elles viennent.

Ce qu'on veut savoir	Source	Mode d'accès
Taille et rentabilité de Tesla, Ford, GM	SEC, via `data/entreprises.csv`	fichier CSV local
CO□ par habitant, par pays	Our World in Data	URL d'un fichier
Ce que le marché pense de Tesla	Yahoo Finance	paquet `yfinance`
Part de l'électricité renouvelable	Banque mondiale	API web, paquet `wbgapi`
États financiers détaillés	SEC EDGAR	API avec en-tête obligatoire
Les données du cours, réunies	`data/strategie.db`	base de données SQL

3.2 Le fichier CSV, et ce qu'il ne dit pas

Un fichier CSV est du texte : une ligne par observation, un séparateur entre les colonnes. Rien dans le fichier n'indique quel est ce séparateur, quel caractère marque la décimale, ni comment les lettres accentuées sont codées. `pd.read_csv` fait des hypothèses raisonnables — virgule, point, UTF-8 — qui conviennent au fichier du cours :

```
entreprises = pd.read_csv("data/entreprises.csv")
entreprises.shape
```

```
(480, 12)
```

Un Excel configuré en français fait d'autres choix : point-virgule entre les colonnes, virgule décimale, encodage Windows-1252. Le fichier `data/entreprises_fr.csv` contient exactement les mêmes données, enregistrées ainsi. Lu avec les hypothèses par défaut, il ne provoque **aucune erreur** :

```
mal = pd.read_csv("data/entreprises_fr.csv", encoding="cp1252")
mal.shape
```

```
(480, 1)
```

Une seule colonne au lieu de douze. pandas n'a trouvé aucune virgule séparatrice et a tout rangé dans la même case. C'est le piège le plus dangereux de l'importation : un échec silencieux. La parade est un réflexe — vérifier `.shape` et `.dtypes` après chaque lecture. Les bons paramètres règlent tout :

```
bon = pd.read_csv("data/entreprises_fr.csv", sep=";", decimal=",", encoding="cp
print(bon.shape)
print(bon[["chiffre_affaires", "resultat_net"]].dtypes)
```

```
(480, 12)
chiffre_affaires    float64
resultat_net        float64
dtype: object
```

Trois autres options servent à lire **moins** : `nrows` pour un aperçu, `usecols` pour ne garder que certaines colonnes, `dtype` pour imposer un type plutôt que de le laisser deviner. Sur les 480 lignes du cours, l'enjeu est nul ; sur un fichier de transactions de plusieurs gigaoctets, c'est la différence entre une analyse et un ordinateur figé (McKinney 2022, ch. 6).

```
leger = pd.read_csv("data/entreprises.csv",
                    usecols=["ticker", "annee", "chiffre_affaires"],
                    dtype={"ticker": "category", "annee": "int16"})
leger.dtypes
```

```
ticker              category
annee                  int16
chiffre_affaires     float64
dtype: object
```

3.3 Lire en ligne, une fois

`pd.read_csv` accepte une URL comme un nom de fichier. Mais lire en ligne à chaque exécution est une mauvaise habitude : le réseau est lent, le fichier distant peut changer sans prévenir, et certains serveurs limitent les requêtes. La règle du cours est de télécharger **une fois** et de relire une copie locale. Une petite fonction suffit :

```
from pathlib import Path

CACHE = Path("data/cache")
CACHE.mkdir(parents=True, exist_ok=True)

def charger(nom, telecharger):
    """Relit data/cache/<nom>.parquet s'il existe ; sinon télécharge, enregi
    fichier = CACHE / f"{nom}.parquet"
    if fichier.exists():
        return pd.read_parquet(fichier)
    df = telecharger()
    df.to_parquet(fichier)
    return df
```

L'argument `telecharger` est une fonction, qui ne s'exécute que si la copie locale manque. Our World in Data publie les émissions de CO□ de tous les pays dans un seul fichier de 79 colonnes ; on n'en garde que cinq :

```
URL_CO2 = "https://raw.githubusercontent.com/owid/co2-data/master/owid-co2-d
co2 = charger("owid_co2", lambda: pd.read_csv(
    URL_CO2, usecols=["country", "year", "iso_code", "co2_per_capita", "shar

pays = ["USA", "CAN", "CHN", "DEU", "NOR", "POL"]
co2[co2["iso_code"].isin(pays) & (co2["year"] == 2023)]
```

	country	year	iso_code	co2_per_capita	share_global_co2
8876	Canada	2023	CAN	13.880	1.432
9932	China	2023	CHN	8.556	31.953
18318	Germany	2023	DEU	7.023	1.559
34909	Norway	2023	NOR	7.043	0.102
37609	Poland	2023	POL	7.307	0.744
48006	United States	2023	USA	14.319	12.911

> **Le piège des certificats sur macOS.** Avec le Python de python.org, `pd.read_csv(url)` peut échouer avec `CERTIFICATE_VERIFY_FAILED` alors

> que `requests.get(url)` fonctionne : `requests` apporte ses propres certificats, pandas utilise ceux du système. Le correctif est d'exécuter une fois `Install Certificates.command`, dans le dossier Applications/Python 3.x.

Les pages web contiennent aussi des tableaux. `pd.read_html` les renvoie tous, sous forme de liste ; il faut un `User-Agent` pour que Wikipédia accepte la requête :

```
URL_WIKI = "https://en.wikipedia.org/wiki/List_of_largest_automotive_companies_
constructeurs = charger("wiki_constructeurs", lambda: pd.read_html(
    URL_WIKI, storage_options={"User-Agent": "Mozilla/5.0"})[0])
constructeurs.head(5)
```

	No.	Company	Industry	Revenue (in billion US$)	Headquarters
0	1	Apple	Electronics	391.035	United States
1	2	Volkswagen Group	Automotive	351.093	Germany
2	3	Toyota	Automotive	315.110	Japan
3	4	Samsung Electronics	Electronics	220.637	South Korea
4	5	Foxconn	Electronics	213.699	Taiwan

Wikipédia sert à **trouver** une source, non à la citer : ces chiffres recopient des rapports annuels, et c'est le rapport annuel qu'un travail sérieux doit citer.

3.4 Les paquets qui parlent aux sources

Certaines sources ont un paquet Python qui cache les détails de l'accès. `yfinance` lit les cours boursiers de Yahoo Finance ; `auto_adjust=True` corrige les cours pour les dividendes et les fractionnements d'actions.

```
import yfinance as yf
from plotstyle import setup
plt = setup()

prix = charger("cours_auto", lambda: yf.download(
    ["TSLA", "F", "GM"], start="2020-01-01", end="2025-01-01",
    auto_adjust=True, progress=False)["Close"])

base100 = prix / prix.iloc[0] * 100
fig, ax = plt.subplots()
for t in base100.columns:
    ax.plot(base100.index, base100[t], label=t)
ax.set_yscale("log")
ax.legend(frameon=False)
plt.show()
```

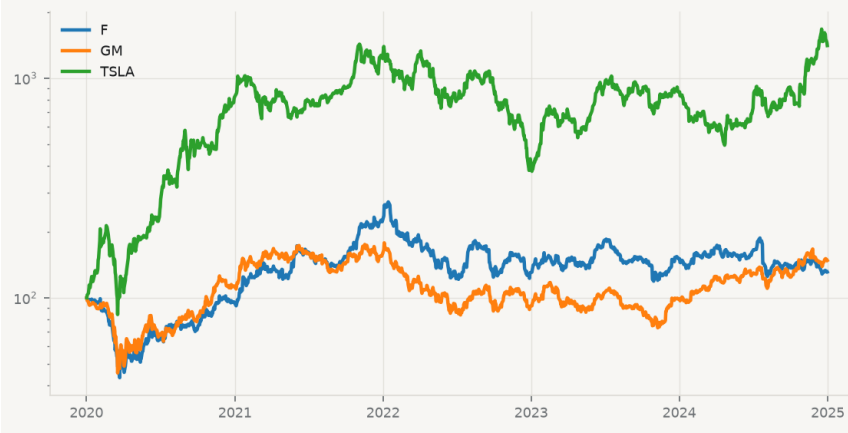


Figure 3.1. Cours ajustés de Tesla, Ford et General Motors, base 100 au premier jour de cotation de 2020, échelle logarithmique. Source : Yahoo Finance via yfinance.

```
rendements = prix.resample("YE").last().pct_change().dropna()
rendements.round(2)
```

Ticker Date	F	GM	TSLA
2021-12-31	1.37	0.41	0.50
2022-12-31	-0.42	-0.42	-0.65
2023-12-31	0.16	0.08	1.02
2024-12-31	-0.13	0.50	0.63

Le rendement annuel nuance déjà la manchette « le marché croit en Tesla » : en 2021, l'action de Ford a progressé davantage que celle de Tesla. `yfinance` n'est pas un produit officiel, et les conditions de Yahoo interdisent de redistribuer ces données ; pour une recherche publiée, les bases de référence sont CRSP et Compustat.
Le paquet `wbgapi` fait de même pour la Banque mondiale :

```
import wbgapi as wb
co2_hab = charger("wb_co2_hab", lambda: wb.data.DataFrame(
    "EN.GHG.CO2.PC.CE.AR5", ["USA", "CAN", "CHN", "NOR", "DEU"],
    time=range(2018, 2023)).reset_index())
co2_hab.round(1)
```

	economy	YR2018	YR2019	YR2020	YR2021	YR2022
0	CAN	16.5	16.2	14.4	14.7	14.8
1	CHN	8.2	8.4	8.5	8.9	8.9
2	DEU	9.0	8.4	7.7	8.1	7.9
3	NOR	8.8	8.5	8.1	8.2	8.0
4	USA	15.6	15.0	13.5	14.3	14.4

3.5 Les API

Une API web est une adresse qui répond avec des données plutôt qu'avec une page. La requête précise une ressource (quels pays, quel indicateur), des paramètres (années, format) et parfois des en-têtes (qui demande, avec quelle clé). La réponse porte un **code de statut** — `200` pour un succès, `404` pour une ressource introuvable, `429` pour trop de requêtes — et, le plus souvent, du JSON : des listes et des dictionnaires imbriqués.

```
import requests

url = "https://api.worldbank.org/v2/country/CAN;USA;CHN;NOR;POL/indicator/EG
reponse = requests.get(url, params={"format": "json", "date": "2015:2022", "
                       timeout=30)
print(reponse.status_code)
meta, observations = reponse.json()
print(meta)
```

```
200
{'page': 1, 'pages': 1, 'per_page': 100, 'total': 40, 'sourceid': '2', 'lastupda
```

La Banque mondiale renvoie une liste de deux éléments : les métadonnées, puis les observations. Une compréhension de liste les range dans un tableau :

```
elec = pd.DataFrame([{"iso3": o["countryiso3code"], "annee": int(o["date"]),
                      "renouv_pct": o["value"]} for o in observations])
elec.pivot(index="annee", columns="iso3", values="renouv_pct").round(1)
```

iso3 annee	CAN	CHN	NOR	POL	USA
2015	64.1	23.9	97.7	14.2	13.6
2016	65.0	25.1	97.8	14.0	15.3
2017	66.6	25.0	97.8	14.4	17.3
2018	65.9	25.8	97.8	13.0	17.2
2019	65.9	26.9	97.8	16.0	17.9
2020	67.1	28.1	98.4	18.4	19.9
2021	67.0	28.4	99.1	17.5	20.3
2022	NaN	NaN	NaN	NaN	NaN

Le tableau répond à la troisième question de la séance 1 : une voiture électrique branchée au réseau norvégien ou canadien roule à l'électricité renouvelable bien plus qu'en Chine, en Pologne ou aux États-Unis. Il montre aussi une limite : la dernière ligne est vide, parce que la série publiée s'arrête en 2021. Une source a toujours un retard, et une analyse doit le dire.

Quand une réponse dépasse une page, l'API l'indique dans ses métadonnées (`pages`) et il faut boucler. La SEC, elle, exige que chaque programme s'identifie par un `User-Agent` qui contient un nom et un courriel. On le lit dans une variable d'environnement, pour qu'il n'apparaisse jamais dans un code partagé :

```
ua = os.environ.get("SEC_USER_AGENT", "HEC Montreal MGMT60XXX course")

def faits_tesla():
    r = requests.get("https://data.sec.gov/api/xbrl/companyfacts/CIK0001318605
                     headers={"User-Agent": ua}, timeout=60)
    r.raise_for_status()
    ventes = r.json()["facts"]["us-gaap"]["RevenueFromContractWithCustomerExclu
    return pd.DataFrame(ventes["units"]["USD"])

brut = charger("sec_tesla_ventes", faits_tesla)
annuel = brut[brut["frame"].fillna("").str.fullmatch(r"CY\d{4}")]
annuel.assign(ventes_milliards=annuel["val"] / 1e9)[["frame", "ventes_milliards
```

	frame	ventes_milliards	form
10	CY2022	81.462	10-K
20	CY2023	96.773	10-K
32	CY2024	97.690	10-K
43	CY2025	94.827	10-K

Une même année de ventes figure dans plusieurs dépôts successifs ; le champ `frame` désigne la valeur retenue pour chaque année civile. C'est la logique de `scripts/build_course_data.py`, qui a construit `data/entreprises.csv`.
Les clés d'API obéissent à la même règle que le courriel : elles vivent dans une variable d'environnement (`os.environ["FRED_API_KEY"]`), jamais dans un fichier versionné. Une clé publiée sur GitHub est repérée par des robots en quelques minutes ; il faut alors la révoquer.

3.6 Les bases de données

Dans une entreprise, les données vivent rarement dans des fichiers : elles sont dans des bases, qu'on interroge en SQL. Le fichier `data/strategie.db` est une base SQLite qui contient les trois tables du cours. `pd.read_sql` envoie la requête et renvoie un DataFrame.

```
import sqlite3
con = sqlite3.connect("data/strategie.db")

pd.read_sql("""
```

```
SELECT secteur,
       COUNT(DISTINCT ticker)                                    AS n,
       ROUND(SUM(chiffre_affaires) / 1000, 1)                    AS ventes_mi
       ROUND(AVG(resultat_net * 1.0 / chiffre_affaires) * 100, 1) AS marge_m
FROM entreprises
WHERE annee = 2023
GROUP BY secteur
ORDER BY ventes_milliards DESC
""", con)
```

	secteur	n	ventes_milliards	marge_moy_pct
0	Commerce de détail	4	1567.1	3.5
1	Technologie	7	1214.5	25.2
2	Énergie	3	590.0	14.6
3	Automobile	3	430.6	8.1
4	Biens de consommation	4	270.6	15.6
5	Pharmaceutique	4	230.3	15.4
6	Industrie	4	202.5	0.3
7	Restauration	2	61.5	22.3
8	Médias et divertissement	1	33.7	16.0

Une jointure relie deux tables par leurs **clés** — ici le pays du siège et l'année :

```
pd.read_sql("""
SELECT e.entreprise, e.chiffre_affaires / 1000 AS ventes_milliards, p.pib_mi
       ROUND(e.chiffre_affaires / 1000 / p.pib_milliards_usd * 100, 2) AS pc
FROM entreprises AS e
JOIN pays AS p ON p.iso3 = e.pays_siege AND p.annee = e.annee
WHERE e.ticker IN (?, ?, ?) AND e.annee = ?
""", con, params=("TSLA", "AAPL", "WMT", 2023))
```

	entreprise	ventes_milliards	pib_milliards_usd	pct_pib
0	Tesla	96.773	27811.52	0.35
1	Apple	383.285	27811.52	1.38
2	Walmart	642.637	27811.52	2.31

Les valeurs filtrées passent par `params=` et des `?`, jamais par du texte collé dans la requête : un nom comme « McDonald's » casserait la requête, et un texte malveillant pourrait la détourner.

```
con.close()
```

DuckDB va plus loin : il interroge un fichier CSV ou parquet **comme une table**, sans base à créer.

```
import duckdb
duckdb.sql("""
SELECT ticker, chiffre_affaires, resultat_net
FROM 'data/entreprises.csv' WHERE annee = 2024
ORDER BY chiffre_affaires DESC LIMIT 5
""").df()
```

	ticker	chiffre_affaires	resultat_net
0	WMT	674538.0	19436.0
1	AMZN	637959.0	59248.0
2	AAPL	391035.0	93736.0
3	GOOGL	350018.0	100118.0
4	XOM	349585.0	33680.0

3.7 Sauvegarder et documenter

Le format parquet conserve les types et occupe moins de place ; le CSV reste lisible par tous. Le cours garde ses copies de travail en parquet et remet ses résultats en CSV.

```
Path("sorties").mkdir(exist_ok=True)
co2.to_csv("sorties/co2.csv", index=False)
co2.to_parquet("sorties/co2.parquet")
{f: round(Path(f).stat().st_size / 1e6, 2) for f in ["sorties/co2.csv", "sortie
```

```
{'sorties/co2.csv': 1.36, 'sorties/co2.parquet': 0.17}
```

Reste l'essentiel, qui ne s'écrit pas en Python : la **fiche de provenance**. Pour chaque source, le producteur, le mode d'accès, la date d'accès, l'unité, la licence et la dernière année disponible. Elle rend le travail vérifiable par un tiers, et par soi-même six semaines plus tard. La séance 4 assemblera ces sources en un seul tableau, ce qui suppose de savoir exactement ce que chacune mesure (Wickham 2014).

3.8 À retenir

— Après chaque import, vérifier `.shape` et `.dtypes` : l'échec le plus grave est silencieux.
— `sep`, `decimal`, `encoding` règlent presque tous les CSV ; `usecols`, `dtype`, `nrows` allègent.
— Télécharger une fois, relire en local : le cache rend le travail rapide et reproductible.

— Une API répond par un code de statut et du JSON ; on vérifie le premier avant de lire le second.
— Courriels et clés d'API vivent dans des variables d'environnement, jamais dans le code.
— SQL sélectionne, filtre, agrège et joint dans la base ; `params=` protège la requête.
— Chaque source a un producteur, une licence et un retard : la fiche de provenance les consigne.

Chapitre 4

Manipuler les données : nettoyer, joindre, remodeler, agréger

4.1 Ce que l'on ne voit pas dans un article

Un article de stratégie présente un tableau de statistiques descriptives, puis des régressions. Il ne montre presque jamais ce qui a précédé : des semaines passées à corriger des types, à harmoniser des identifiants, à joindre des sources qui ne parlent pas la même langue, et à décider quoi faire des trous. Ce travail n'est pas préliminaire. C'est là que se prennent la plupart des décisions qui déterminent le résultat — et la plupart des erreurs qui le compromettent.

Ce chapitre part de deux questions que se pose un gestionnaire. La première : *qui sont mes vrais concurrents ?* Comparer une entreprise à la moyenne de toutes les autres n'a guère de sens ; il faut la comparer à son secteur, la même année. La seconde : *les fusions-acquisitions créent-elles de la valeur ?* Pour en avoir un premier aperçu, il faut importer des cours boursiers, calculer des rendements entreprise par entreprise, les aligner sur ceux du marché et découper une fenêtre autour de l'annonce. Aucune de ces deux questions ne demande de statistique avancée. Toutes deux demandent de savoir **manipuler** des tables.

4.2 Nettoyer

Les données arrivent rarement prêtes. Voici une petite table comme on en reçoit d'un collègue ou d'un tableur :

```
brut = pd.DataFrame({
    "Entreprise ": [" Tesla", "Ford", "ford", "General Motors", "Ford"],
    "Annee":       ["2023", "2023", "2023", "2023", "2023"],
    "Ventes (M$)": ["96 773", "176 191", "176 191", "157 658", "176 191"],
    "R&D":         ["3 969", "8 200", "8 200", "n.d.", "8 200"],
})
brut.dtypes
```

```
Entreprise     str
Annee          str
Ventes (M$)    str
R&D            str
dtype: object
```

Tout y est du texte : on ne peut ni additionner ni comparer. Le nettoyage suit toujours le même ordre — les **noms**, le **texte**, les **types**, puis les **doublons** — parce

que chaque étape dépend de la précédente : tant que `"ford"` et `"Ford"` coexistent, aucune recherche de doublons ne les trouvera.

```
propre = brut.rename(columns={"Entreprise ": "entreprise", "Annee": "annee",
                              "Ventes (M$)": "ventes", "R&D": "rd"})
propre["entreprise"] = propre["entreprise"].str.strip().str.title()
propre["annee"] = propre["annee"].astype(int)
propre["ventes"] = propre["ventes"].str.replace(" ", "").astype(float)
propre["rd"] = pd.to_numeric(propre["rd"].str.replace(" ", ""), errors="coerce"
print(propre.duplicated().sum(), "doublon(s)")
propre = propre.drop_duplicates()
propre
```

```
2 doublon(s)
```

	entreprise	annee	ventes	rd
0	Tesla	2023	96773.0	3969.0
1	Ford	2023	176191.0	8200.0
3	General Motors	2023	157658.0	NaN

Deux outils méritent attention. `astype(float)` échoue dès qu'une cellule n'est pas un nombre ; c'est utile quand on s'attend à des nombres partout. `pd.to_numeric(..., errors="coerce")` transforme au contraire toute valeur illisible en `NaN` : c'est commode, mais cela cache le problème si l'on ne compte pas ensuite les valeurs manquantes créées.

4.2.1 Les valeurs manquantes

Dans les données du cours, il n'y a aucun doublon entreprise × année, mais beaucoup de trous :

```
entreprises = pd.read_csv("data/entreprises.csv")
print(entreprises.duplicated(["ticker", "annee"]).sum(), "doublon entreprise ×
entreprises.isna().sum()
```

```
0 doublon entreprise × année
```

```
ticker               0
entreprise           0
cik                  0
secteur              0
pays_siege           0
annee                0
chiffre_affaires     3
```

```
resultat_net          11
actifs_totaux          5
capitaux_propres       3
rd                   135
investissements      121
dtype: int64
```

On a trois options : **retirer** les lignes (`dropna`), **remplir** (`fillna`) ou **signaler** (créer une colonne booléenne). La troisième devrait toujours précéder les deux autres. La colonne `rd` montre pourquoi :

```
entreprises["rd_manquant"] = entreprises["rd"].isna()
(entreprises.groupby("entreprise")["rd_manquant"].mean()
    .loc[lambda s: s > 0].sort_values(ascending=False).round(2))
```

```
entreprise
Amazon              1.00
Coca-Cola           1.00
Costco              1.00
McDonald's          1.00
Nike                1.00
Starbucks           1.00
Target              1.00
Walmart             1.00
General Motors      0.73
Alphabet            0.20
Procter & Gamble    0.07
Name: rd_manquant, dtype: float64
```

Les entreprises qui ne publient jamais de ligne « recherche et développement » sont des distributeurs, des chaînes de restauration et des marques de consommation — mais aussi Amazon, qui investit massivement en technologie sous un autre intitulé. Remplir par zéro rangerait Amazon parmi les entreprises les moins innovantes de l'échantillon. L'absence de valeur est ici une information sur la **présentation des comptes**, pas sur l'effort d'innovation.

4.3 Joindre

Une analyse combine presque toujours plusieurs sources. La jointure (`merge`) associe les lignes de deux tables qui partagent une **clé**. Le risque principal est silencieux : si la clé apparaît plusieurs fois du côté où l'on attendait une seule ligne, chaque observation est dupliquée, et rien ne signale l'erreur. Deux arguments de `merge` la rendent visible : `validate="m:1"` exige que la table de droite ait au plus une ligne par clé, et `indicator=True` ajoute une colonne `_merge` qui dit d'où vient chaque ligne.

```
pays = pd.read_csv("data/pays.csv")
avant = len(entreprises)
ent_pays = entreprises.merge(
    pays[["iso3", "annee", "pib_hab_usd", "croissance_pib"]],
    left_on=["pays_siege", "annee"], right_on=["iso3", "annee"],
    how="left", validate="m:1", indicator=True,
)
print(avant, "→", len(ent_pays), "lignes")
ent_pays.groupby("annee")["_merge"].value_counts().unstack().tail(3)
```

```
480 → 480 lignes
```

_merge annee	left_only	right_only	both
2022	0	0	32
2023	0	0	32
2024	32	0	0

Le nombre de lignes n'a pas changé, ce qui confirme la relation plusieurs-à-un. Les lignes sans partenaire sont toutes en 2024 : les données de la Banque mondiale du cours s'arrêtent en 2023. Une jointure `how="inner"` aurait supprimé une année entière sans le dire ; `how="left"` avec `indicator=True` la montre.
La table des secteurs se joint de la même façon, par la colonne `secteur` :

```
secteurs = pd.read_csv("data/secteurs.csv")
entreprises.merge(secteurs, on="secteur", how="left", validate="m:1").shape
```

```
(480, 14)
```

`pd.concat`, enfin, n'associe pas des colonnes mais **empile** des lignes : c'est l'outil pour réunir des fenêtres de rendements calculées séparément, comme plus bas.

4.4 Remodeler

Wickham (2014) appelle **rangées** (*tidy*) les données où chaque variable est une colonne, chaque observation une ligne, et chaque type d'unité une table. `entreprises.csv` est rangé : une ligne correspond à une entreprise et une année. Beaucoup de sources publient pourtant un format **large**, avec une colonne par année — c'est le format par défaut des téléchargements de la Banque mondiale et de la plupart des tableaux Excel.

```
large = (pays[pays["iso3"].isin(["CAN", "USA", "MEX", "CHN"]) & (pays["annee"]
         .pivot(index="pays", columns="annee", values="pib_hab_usd").round(0))
large
```

annee pays	2019	2020	2021	2022	2023
Canada	46353.0	43538.0	52887.0	56497.0	54848.0
China	10343.0	10627.0	12887.0	12971.0	12951.0
Mexico	10370.0	8841.0	10314.0	11406.0	13831.0
United States	65228.0	64465.0	71441.0	78009.0	82587.0

Ce format se lit bien, mais il ne se prête ni à un `groupby` par année ni à un graphique. `melt` le remet au format long :

```
long = large.reset_index().melt(id_vars="pays", var_name="annee", value_name
long.shape
```

```
(20, 3)
```

`pivot` refuse les combinaisons index × colonne en double ; `pivot_table` les agrège avec la fonction qu'on lui donne. La médiane de la marge nette par secteur et par année s'obtient ainsi en une instruction :

```
(entreprises
 .assign(marge=entreprises["resultat_net"] / entreprises["chiffre_affaires"]
 .pivot_table(index="secteur", columns="annee", values="marge", aggfunc="med
 .loc[:, [2019, 2020, 2021, 2022, 2023]].round(3))
```

annee secteur	2019	2020	2021	2022	2023
Automobile	0.001	0.023	0.105	0.067	0.062
Biens de consommation	0.088	0.156	0.159	0.143	0.145
Commerce de détail	0.029	0.036	0.046	0.022	0.032
Industrie	0.127	0.109	0.144	0.126	0.063
Médias et divertissement	0.093	0.110	0.172	0.142	0.160
Pharmaceutique	0.312	0.199	0.259	0.235	0.098
Restauration	0.209	0.143	0.235	0.184	0.223
Technologie	0.256	0.260	0.295	0.199	0.253
Énergie	0.056	-0.126	0.100	0.151	0.109

4.5 Agréger, et comparer chaque entreprise à son groupe

`groupby` découpe la table en groupes. Ce que l'on fait ensuite détermine la forme du résultat : `agg` **réduit** chaque groupe à une ligne, `transform` **renvoie** une valeur pour chaque ligne d'origine. Le premier produit un tableau de synthèse, le second permet de comparer chaque observation à son groupe.

```
(entreprises[entreprises["annee"] == 2023]
 .groupby("secteur")
 .agg(n=("ticker", "nunique"),
      ventes_mediane=("chiffre_affaires", "median"),
      rd_publiee=("rd", "count"))
 .sort_values("ventes_mediane", ascending=False))
```

secteur	n	ventes_mediane	rd_publiee
Commerce de détail	4	408537.5	0
Énergie	3	196913.0	3
Automobile	3	157658.0	2
Technologie	7	134902.0	7
Biens de consommation	4	66684.0	2
Pharmaceutique	4	55514.5	4
Industrie	4	50034.5	4
Médias et divertissement	1	33723.3	1
Restauration	2	30734.8	0

Pour répondre à la question des vrais concurrents, on calcule le rendement des actifs (ROA), puis la moyenne du ROA du secteur **la même année**, et l'écart entre les deux :

```
entreprises["roa"] = entreprises["resultat_net"] / entreprises["actifs_totaux"]
entreprises["roa_secteur"] = entreprises.groupby(["secteur", "annee"])["roa"].t
entreprises["ecart_secteur"] = entreprises["roa"] - entreprises["roa_secteur"]

classement = (entreprises[entreprises["annee"] == 2023]
              [["entreprise", "secteur", "roa", "roa_secteur", "ecart_secteur"]
              .sort_values("ecart_secteur", ascending=False).round(3))
pd.concat([classement.head(4), classement.tail(4)])
```

	entreprise	secteur	roa	roa_secteur	ecart_secteur
118	Nvidia	Technologie	0.453	0.186	0.267
238	Johnson & Johnson	Pharmaceutique	0.210	0.076	0.134
433	Caterpillar	Industrie	0.118	0.014	0.104
58	Apple	Technologie	0.274	0.186	0.089
253	Merck & Co.	Pharmaceutique	0.003	0.076	-0.073
148	IBM	Technologie	0.055	0.186	-0.130
448	3M	Industrie	-0.138	0.014	-0.152
133	Intel	Technologie	0.009	0.186	-0.177

entreprise	secteur	roa	roa_secteur	ecart_secteur

```
d = classement.sort_values("ecart_secteur")
fig, ax = plt.subplots(figsize=(7.2, 6.4))
couleurs = [CL.good if v > 0 else CL.warn for v in d["ecart_secteur"]]
ax.barh(d["entreprise"], d["ecart_secteur"] * 100, color=couleurs)
ax.axvline(0, color=CL.muted, lw=0.8)
ax.set_xlabel("points de pourcentage")
ax.tick_params(axis="y", labelsize=8)
plt.show()
```

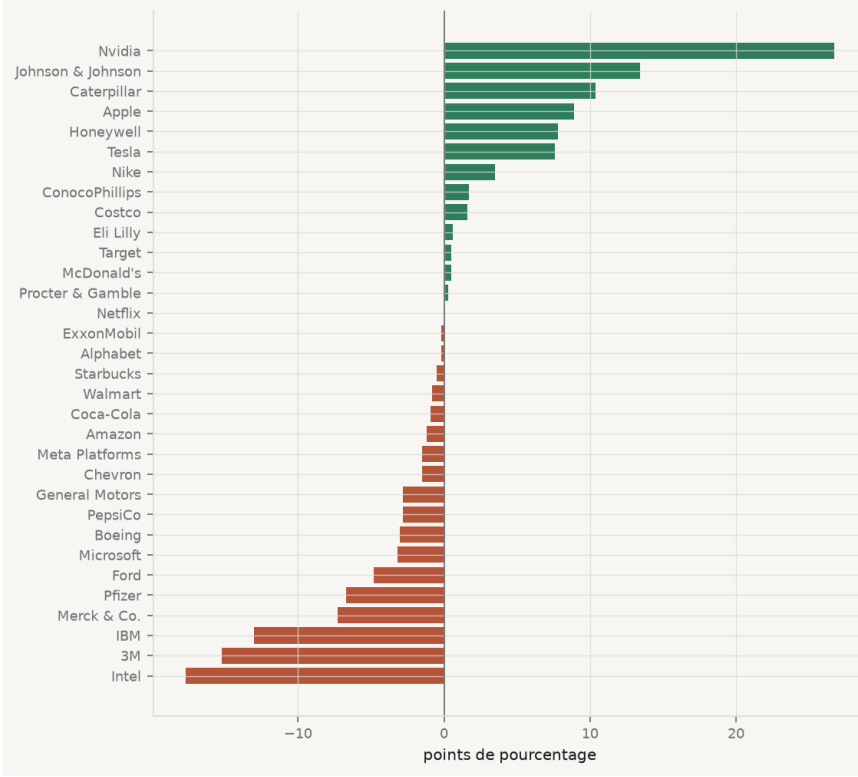


Figure 4.1. Écart du rendement des actifs à la moyenne du secteur, 2023. Une barre à droite de zéro signale une entreprise plus rentable que ses concurrents directs la même année.

Le choix du groupe de comparaison change le diagnostic. Microsoft affiche un ROA supérieur à celui de la plupart des 32 entreprises, mais inférieur à la moyenne de son propre secteur en 2023, tiré vers le haut par Nvidia et Apple. Inversement, une

moyenne calculée sur un très petit groupe ne dit presque rien : Netflix est seule dans son secteur, et son écart y vaut zéro par construction.

```
entreprises.groupby("secteur")["ticker"].nunique().sort_values()
```

```
secteur
Médias et divertissement    1
Restauration                2
Automobile                  3
Énergie                     3
Biens de consommation       4
Commerce de détail          4
Industrie                   4
Pharmaceutique              4
Technologie                 7
Name: ticker, dtype: int64
```

4.6 Le temps, entité par entité

Dans un panel, toute opération qui regarde « l'année précédente » doit être faite **par entité**, après un tri. Sans `groupby("ticker")`, `shift` et `pct_change` iraient chercher la dernière année d'une entreprise pour calculer la croissance de la suivante.

```
entreprises = entreprises.sort_values(["ticker", "annee"])
entreprises["croissance"] = entreprises.groupby("ticker")["chiffre_affaires"].p
entreprises["croissance_3ans"] = (entreprises.groupby("ticker")["croissance"]
                                  .transform(lambda s: s.rolling(3).mean()))
(entreprises[entreprises["ticker"] == "TSLA"]
 [["annee", "chiffre_affaires", "croissance", "croissance_3ans"]].tail(6).round
```

	annee	chiffre_affaires	croissance	croissance_3ans
9	2019	24578.0	0.145	0.550
10	2020	31536.0	0.283	0.418
11	2021	53823.0	0.707	0.378
12	2022	81462.0	0.514	0.501
13	2023	96773.0	0.188	0.469
14	2024	97690.0	0.009	0.237

La moyenne mobile sur trois ans lisse les à-coups d'une année exceptionnelle et fait apparaître la tendance : la croissance de Tesla ralentit nettement après 2022.

4.7 Les fusions-acquisitions créent-elles de la valeur ? Un premier regard

Le 18 janvier 2022, Microsoft annonce l'acquisition d'Activision Blizzard ; le 13 mars 2023, Pfizer annonce celle de Seagen. Si le marché juge qu'une acquisition détruit de la valeur pour l'acquéreur, le cours de ce dernier devrait baisser **plus que le marché** autour de l'annonce.
On commence par importer les cours une seule fois, en les gardant dans un cache local :

```
from pathlib import Path
import yfinance as yf

def cours(ticker, debut="2021-06-01", fin="2023-12-31"):
    """Cours de clôture ajustés, mis en cache dans data/cache/ au premier ap
    fichier = Path("data/cache") / f"{ticker}.parquet"
    if not fichier.exists():
        fichier.parent.mkdir(parents=True, exist_ok=True)
        d = yf.download(ticker, start=debut, end=fin, progress=False, auto_a
        d.columns = [c[0] if isinstance(c, tuple) else c for c in d.columns]
        d.to_parquet(fichier)
    return pd.read_parquet(fichier)["Close"]

prix = pd.DataFrame({t: cours(t) for t in ["MSFT", "PFE", "SPY"]})
rendements = prix.pct_change().dropna()
rendements.describe().loc[["mean", "std"]].round(4)
```

	MSFT	PFE	SPY
mean	0.0008	-0.0002	0.0003
std	0.0179	0.0168	0.0114

Le fonds `SPY`, qui réplique le S&P 500, sert de « marché ». Le **rendement anormal naïf** est la différence entre le rendement de l'acquéreur et celui du marché ; on le cumule sur une fenêtre de cinq jours de Bourse de part et d'autre de l'annonce.

```
def fenetre(ticker, annonce, avant=5, apres=5):
    r = rendements[[ticker, "SPY"]].copy()
    r["anormal"] = r[ticker] - r["SPY"]
    i = r.index.searchsorted(pd.Timestamp(annonce))   # jour de Bourse de l'
    w = r.iloc[i - avant : i + apres + 1].copy()
    w["jour"] = range(-avant, apres + 1)
    w["cumul"] = w["anormal"].cumsum()
    return w
```

```
fen = pd.concat([fenetre("MSFT", "2022-01-18").assign(operation="Microsoft · Ac
                 fenetre("PFE", "2023-03-13").assign(operation="Pfizer · Seagen
fen.pivot(index="jour", columns="operation", values="cumul").round(4)
```

jour \ operation	Microsoft · Activision	Pfizer · Seagen
-5	0.0020	-0.0017
-4	-0.0049	-0.0051
-3	0.0029	-0.0122
-2	-0.0257	-0.0102
-1	-0.0084	0.0025
0	-0.0150	0.0159
1	-0.0024	0.0003
2	0.0030	0.0161
3	0.0042	0.0008
4	0.0011	0.0058
5	-0.0133	0.0139

```
fig, ax = plt.subplots()
for (nom, g), c in zip(fen.groupby("operation"), [CL.accent, CL.warn]):
    ax.plot(g["jour"], g["cumul"] * 100, marker="o", color=c, label=nom)
ax.axvline(0, color=CL.muted, lw=1)
ax.axhline(0, color=CL.muted, lw=0.8)
ax.set_xlabel("jours de Bourse autour de l'annonce")
ax.set_ylabel("%")
ax.legend(frameon=False)
plt.show()
```

Les deux courbes vont dans des directions opposées, et restent de l'ordre d'un ou deux points de pourcentage — à peine plus que l'écart-type d'un seul rendement quotidien de ces titres, qui est lui-même voisin de deux pour cent. Il serait donc imprudent d'en tirer quoi que ce soit.

Ce calcul n'est **pas** une étude d'événement. Une étude d'événement en bonne et due forme (MacKinlay 1997) estime d'abord, sur une période antérieure à l'annonce, la relation normale entre le titre et le marché (le modèle de marché), plutôt que de supposer qu'ils évoluent un pour un ; elle porte sur un grand nombre d'annonces, dont elle fait la moyenne ; elle teste statistiquement si le rendement anormal moyen diffère de zéro ; et elle s'interroge sur les fuites d'information avant l'annonce et sur les autres nouvelles publiées le même jour. Le code ci-dessus est correct ; c'est la conclusion qu'on serait tenté d'en tirer qui ne le serait pas. Les séances consacrées aux fusions-acquisitions reprendront la question avec la méthode complète.

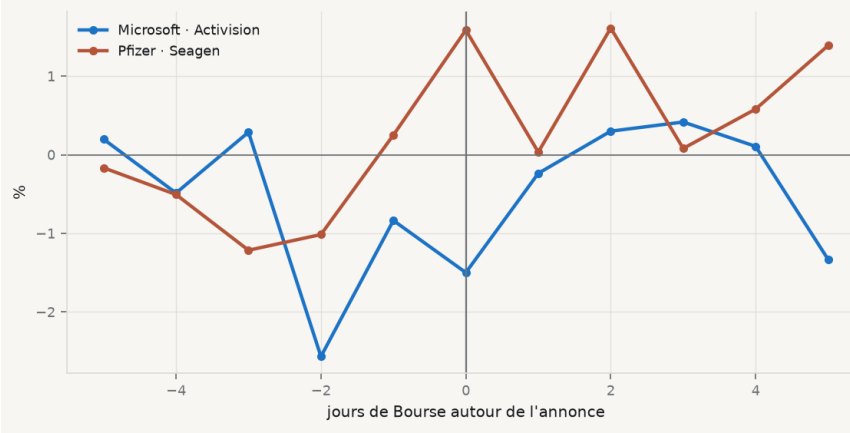


Figure 4.2. Rendement anormal cumulé naïf (acquéreur moins SPY) autour de deux annonces d'acquisition. Le jour 0 est le premier jour de Bourse à partir de l'annonce.

4.8 À retenir

— Nettoyer suit un ordre : noms, texte, types, doublons. `errors="coerce"` est utile à condition de compter les `NaN` qu'il crée.
— Une valeur manquante se **signale** avant d'être retirée ou remplie ; elle est souvent une information.
— Chaque jointure se contrôle : nombre de lignes avant et après, `validate=`, `indicator=`.
— Les données rangées — une ligne par observation — sont le format de travail ; `melt` et `pivot` font l'aller-retour avec le format large des sources.
— `agg` résume un groupe ; `transform` compare chaque observation à son groupe. Le choix du groupe fait partie de l'analyse.
— Dans un panel, `shift`, `pct_change` et `rolling` s'appliquent **par entité**, après un tri.

Chapitre 5

Représenter les données

5.1 Une manchette, plusieurs vérités

« L'Amérique crée plus de valeur que tout autre pays. » La phrase circule, et elle est vraie — pour une certaine définition de *valeur*. Si la valeur est la production totale, les États-Unis dominent. Si elle est la production par habitant, ou la croissance, ou la capacité à attirer l'investissement étranger, le classement change du tout au tout.

```
mesures = {"pib_milliards_usd": "PIB total",
           "pib_hab_usd": "PIB par habitant",
           "croissance_pib": "Croissance du PIB",
           "ide_entrants_pct_pib": "IDE entrants (% du PIB)"}
rangs = pd.DataFrame({
    "Mesure": list(mesures.values()),
    "Rang des États-Unis": [int(p23[m].rank(ascending=False)[p23["iso3"] ==
    "Pays mesurés": [int(p23[m].notna().sum()) for m in mesures],
})
rangs
```

Table 5.1. Le rang des États-Unis en 2023 selon quatre mesures de la « valeur ». Source : Banque mondiale.

	Mesure	Rang des États-Unis	Pays mesurés
0	PIB total	1	204
1	PIB par habitant	11	204
2	Croissance du PIB	113	204
3	IDE entrants (% du PIB)	132	195

Aucune de ces quatre lignes n'est fausse. Le choix de la mesure est un choix d'argument, et le graphique qui suit ce choix le rend visible — ou le cache. Ce chapitre traite de ce passage : de la donnée à l'image, et de l'image à la décision.

5.2 Pourquoi regarder plutôt que résumer

En 1973, le statisticien Francis Anscombe publie quatre petits jeux de onze points (Anscombe 1973). Leurs moyennes, leurs variances et leur corrélation sont identiques à la deuxième décimale. Un tableau de statistiques résumées les déclare interchangeables.

```
x  = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
x4 = [8, 8, 8, 8, 8, 8, 8, 19, 8, 8, 8]
ys = {"I":   [8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68
      "II":  [9.14, 8.14, 8.74, 8.77, 9.26, 8.10, 6.13, 3.10, 9.13, 7.26, 4.74
      "III": [7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73
      "IV":  [6.58, 5.76, 7.71, 8.84, 8.47, 7.04, 5.25, 12.50, 5.56, 7.91, 6.89
fig, axes = plt.subplots(1, 4, figsize=(8, 2.6), sharex=True, sharey=True)
for ax, (jeu, y) in zip(axes, ys.items()):
    xx = x4 if jeu == "IV" else x
    ax.scatter(xx, y, color=CL.ink, s=12)
    b, a = np.polyfit(xx, y, 1)
    ax.plot([3, 20], [a + 3 * b, a + 20 * b], color=CL.accent, lw=1.2)
    ax.set_title(f"Jeu {jeu}  ·  r = {np.corrcoef(xx, y)[0, 1]:.2f}", loc="lef
plt.show()
```

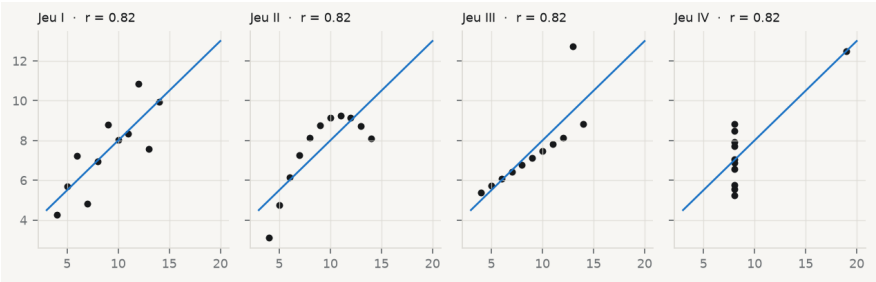


Figure 5.1. Le quartet d'Anscombe : quatre jeux de données aux statistiques résumées identiques, et la même droite de régression.

Le premier jeu est une relation linéaire bruitée ; le deuxième, une courbe ; le troisième, une droite parfaite déviée par un seul point ; le quatrième n'a **aucune** relation, sauf celle qu'un point isolé fabrique. La leçon est toujours valable : on regarde *avant* de résumer, et on résume *ce qu'on a vu*.

5.3 Ce que l'œil lit bien

Un graphique transforme des nombres en propriétés visuelles : une position, une longueur, un angle, une surface, une couleur. Cleveland et McGill ont mesuré, par des expériences, la précision avec laquelle on lit chacune (Cleveland et McGill 1984). L'ordre qu'ils obtiennent est devenu la règle de base de la visualisation :

1. la position sur une échelle commune ;
2. la position sur des échelles non alignées ;
3. la longueur ;
4. l'angle et la pente ;
5. l'aire ;

6. le volume, la saturation et la teinte.

La conséquence pratique est simple. La comparaison la plus importante doit être confiée à l'encodage le plus précis. Le diagramme circulaire encode par l'angle ; la barre, par la position. Les mêmes nombres, présentés des deux façons, ne se lisent pas avec la même précision (Figure 5.2).

```
top5 = p23.nlargest(5, "pib_milliards_usd").set_index("pays")["pib_milliards_
total = p23["pib_milliards_usd"].sum()
parts = pd.concat([top5, pd.Series({"Reste du monde": total - top5.sum()})])
fig, (a1, a2) = plt.subplots(1, 2, figsize=(8, 3))
a1.pie(parts, labels=parts.index, colors=CL.ramp, textprops={"fontsize": 7})
s = parts.sort_values() * 100
a2.barh(s.index, s, color=CL.accent)
for i, v in enumerate(s):
    a2.text(v + 0.5, i, f"{v:.0f} %", va="center", fontsize=7)
a2.set_xlabel("Part du PIB mondial (%)")
plt.show()
print(f"Part des États-Unis : {parts['United States']:.1%}")
```

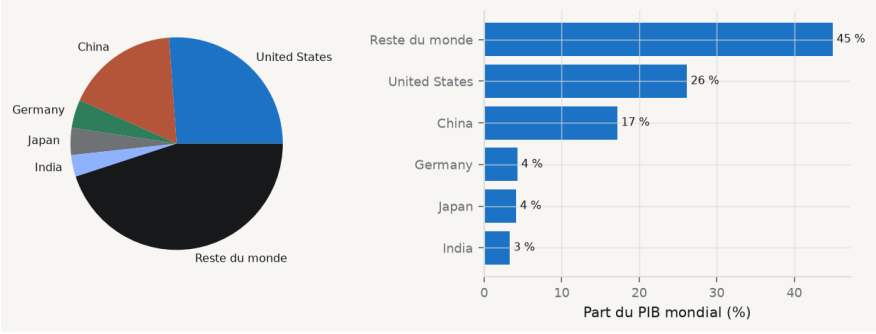


Figure 5.2. Part du PIB mondial en 2023 : les mêmes six nombres, lus par l'angle et par la position. Source : Banque mondiale.

```
Part des États-Unis : 26.1%
```

5.4 L'anatomie d'une figure

En matplotlib, une figure se construit en deux objets : `fig`, la page, et `ax`, le système d'axes où l'on dessine. Écrire `fig, ax = plt.subplots()` puis appeler les méthodes de `ax` plutôt que les fonctions `plt.*` garde chaque instruction attachée à un panneau précis — ce qui devient indispensable dès qu'il y en a plusieurs.
Cinq éléments séparent une figure de travail d'une figure que l'on remet :

- un **titre qui énonce le message** (« L'écart entre les États-Unis et le Canada se creuse »), plutôt que le sujet (« PIB par habitant ») ;
- une **unité** sur chaque axe ;
- la **source**, en petit, sous la figure ;
- un **étiquetage direct** des séries, qui évite l'aller-retour vers une légende ;
- un enregistrement à la bonne résolution : `fig.savefig("sorties/f.png", dpi=200)`, appelé **avant** `plt.show()`.

```
choix = {"USA": CL.warn, "CAN": CL.accent, "JPN": CL.good, "MEX": CL.muted}
fig, ax = plt.subplots(figsize=(7, 3.6))
for iso, coul in choix.items():
    d = pays[pays["iso3"] == iso].dropna(subset=["pib_hab_usd"])
    ax.plot(d["annee"], d["pib_hab_usd"] / 1000, color=coul)
    fin = d.iloc[-1]
    ax.text(fin["annee"] + 0.4, fin["pib_hab_usd"] / 1000, fin["pays"], color=c
            va="center", fontsize=8)
ax.set_xlim(2000, 2027)
ax.set_title("L'écart entre les États-Unis et le Canada se creuse depuis 2014",
ax.set_ylabel("milliers de $ US courants")
plt.show()
ecart = pays[pays["iso3"].isin(["USA", "CAN"]) & pays["annee"].isin([2014, 202
    .pivot(index="annee", columns="iso3", values="pib_hab_usd")
print((ecart["USA"] - ecart["CAN"]).round(0))
```

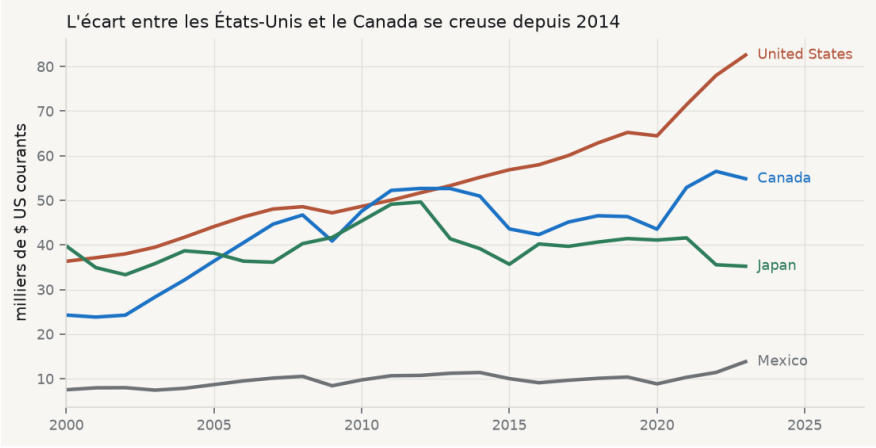


Figure 5.3. PIB par habitant, en milliers de dollars US courants, 2000-2023. Source : Banque mondiale.

annee

```
2014     4192.0
2023    27739.0
dtype: float64
```

5.5 Choisir le graphique selon la question

Le bon graphique se choisit à partir de la question, jamais à partir du menu d'un logiciel. Cinq questions couvrent l'essentiel du travail d'un analyste en stratégie.

Question	Forme	En Python
Comparer des catégories	Barres horizontales triées	`ax.barh`, `sns.barplot`
Suivre une évolution	Ligne	`ax.plot`, `sns.lineplot`
Voir une distribution	Histogramme, boîte à moustaches	`sns.histplot`, `sns.boxplot`
Voir une relation	Nuage de points, souvent en échelle log	`sns.scatterplot`
Montrer une composition	Barres empilées	`df.plot.barh(stacked=True)`

La distribution du PIB par habitant illustre pourquoi l'échelle compte. Sur une échelle linéaire, presque tous les pays s'écrasent dans la première classe ; sur une échelle logarithmique, la forme apparaît (Figure 5.4).

```
fig, (a1, a2) = plt.subplots(1, 2, figsize=(8, 2.8))
sns.histplot(p23["pib_hab_usd"] / 1000, bins=30, ax=a1, color=CL.accent)
a1.set(xlabel="milliers de $", ylabel="pays")
sns.histplot(p23["pib_hab_usd"], bins=30, log_scale=True, ax=a2, color=CL.ac
a2.set(xlabel="$ (échelle log)", ylabel="")
usa = p23.loc[p23["iso3"] == "USA", "pib_hab_usd"].iloc[0]
a2.axvline(usa, color=CL.warn)
plt.show()
print(f"moyenne {p23['pib_hab_usd'].mean():,.0f} $ · médiane {p23['pib_hab_u
      f"pays devant les États-Unis : {(p23['pib_hab_usd'] > usa).sum()}")
```

```
moyenne 22,296 $ · médiane 7,862 $ · pays devant les États-Unis : 10
```

L'écart entre la moyenne et la médiane est à lui seul un diagnostic : quelques économies très riches tirent la moyenne vers le haut. Et plusieurs des pays qui devancent les États-Unis sont de petits territoires — micro-États, places financières — dont le PIB par habitant dit peu de chose sur une stratégie d'implantation.

5.6 seaborn : la statistique en une ligne

seaborn s'appuie sur matplotlib et parle directement le langage des DataFrames : on lui donne `data=`, puis les noms de colonnes pour `x`, `y`, `hue` (la couleur), `col` (les

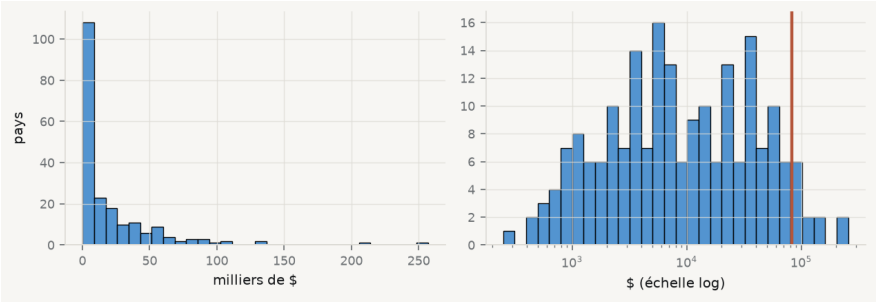


Figure 5.4. Distribution du PIB par habitant en 2023, échelle linéaire et logarithmique. Source : Banque mondiale.

panneaux). La boîte à moustaches par secteur résume quinze ans de marges nettes pour les 32 entreprises du cours (Figure 5.5).

```
ent["marge"] = ent["resultat_net"] / ent["chiffre_affaires"] * 100
ordre = ent.groupby("secteur")["marge"].median().sort_values().index
fig, ax = plt.subplots(figsize=(7.5, 3.6))
sns.boxplot(data=ent, y="secteur", x="marge", order=ordre, color=CL.line, ax=ax
ax.axvline(0, color=CL.warn, lw=1)
ax.set(xlim=(-30, 60), xlabel="Marge nette (%)", ylabel="")
plt.show()
ent.groupby("secteur")["marge"].median().sort_values().round(1)
```

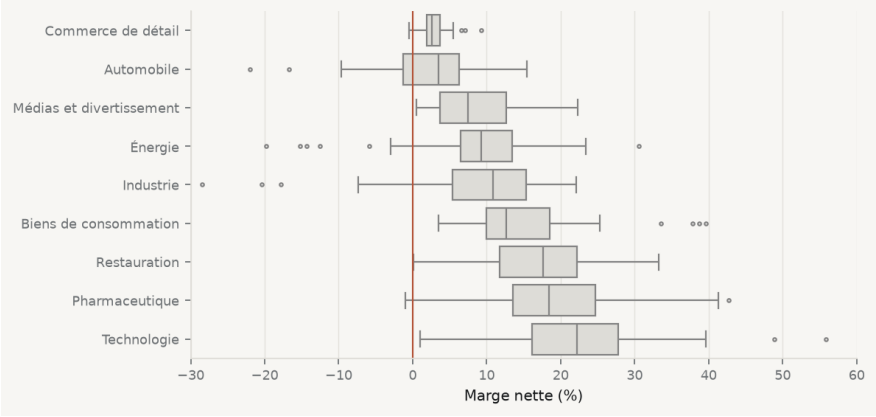
Le trait central est la médiane ; la boîte couvre la moitié centrale des observations ; les points isolés sont des années exceptionnelles. Le commerce de détail travaille sur des marges minces, la technologie et la pharmacie sur des marges épaisses : ce sont deux modèles d'affaires, pas deux niveaux de compétence.

Les **petits multiples** — un panneau par groupe, avec `sns.relplot(..., col="secteur")` — permettent de comparer des formes d'évolution sans superposer trente lignes. Le choix `sharey=True` ou `False` change la lecture : une échelle commune compare les niveaux, une échelle libre compare les formes.

5.7 plotly : explorer avant de conclure

Pour explorer, un graphique interactif vaut mieux qu'un graphique figé : on survole un point pour savoir quel pays il représente, on zoome sur un groupe. `plotly.express` produit ces figures avec la même syntaxe que seaborn.

En survolant les points les plus hauts, on découvre que l'IDE rapporté au PIB est souvent le plus élevé dans de petites économies ou des places financières : la mesure capte autant les flux de passage que l'attractivité productive. C'est une conclusion que l'exploration interactive fait apparaître, et qu'un graphique statique destiné à un rapport devra ensuite énoncer clairement.



(a) Marge nette (résultat net / chiffre d'affaires) par secteur, 32 entreprises, 2010-2024. Source : SEC EDGAR.

```
secteur
Commerce de détail           2.6
Automobile                   3.5
Médias et divertissement     7.4
Énergie                      9.3
Industrie                   10.9
Biens de consommation       12.6
Restauration                17.6
Pharmaceutique              18.4
Technologie                 22.2
Name: marge, dtype: float64
```

(b)

Figure 5.5

5.8 Les graphiques qui trompent sans mentir

Trois procédés reviennent dans la presse comme dans les présentations d'entreprise.

L'axe tronqué. Une barre encode une longueur ; si l'axe commence à 80 plutôt qu'à 0, un écart de dix pour cent paraît un écart du simple au triple. Les lignes, qui encodent une position, peuvent zoomer sans tromper.

Le double axe. Deux séries d'unités différentes sur deux échelles verticales : comme chaque échelle se choisit librement, l'auteur décide où les courbes se croisent. Deux panneaux disent la même chose sans ce pouvoir arbitraire.

Le camembert en trois dimensions. Il cumule l'encodage le moins précis (l'angle) et une perspective qui grossit la part du premier plan. Des barres horizontales triées le remplacent toujours avec profit.

5.9 À retenir

— Une affirmation sur la « valeur » n'est vérifiable qu'une fois la **mesure** choisie, et le graphique rend ce choix visible.
— On **regarde** les données avant de les résumer (Anscombe), et on confie la comparaison principale à la **position** (Cleveland et McGill).
— Une figure remise a un titre-message, des unités, une source et, si possible, un étiquetage direct.
— matplotlib pour le contrôle, seaborn pour la statistique, plotly pour l'exploration.

Chapitre 6

Statistiques de base : décrire, comparer, relier

6.1 Ce que l'on voit, et ce que l'on peut affirmer

Les cinq premiers chapitres ont appris à obtenir des données et à les regarder. Celui-ci pose une question que le regard seul ne tranche pas : **une différence observée est-elle assez solide pour être affirmée ?** Tesla affiche en 2023 une marge nette supérieure à celle de Ford ; les entreprises technologiques semblent plus rentables que les autres ; les grandes dépenses de R-D paraissent aller avec de grands profits. Chacune de ces impressions peut être vraie, ou être un accident de l'échantillon que l'on a sous les yeux.

La statistique ne supprime pas cette incertitude. Elle la **mesure**, et elle oblige à dire, avec chaque résultat, de combien il pourrait être faux. C'est une discipline plus qu'une technique : la plupart des erreurs de ce chapitre ne sont pas des erreurs de calcul, mais des phrases qui affirment plus que les chiffres ne le permettent.

Le chapitre travaille sur trois indicateurs construits à partir de `data/entreprises.csv` : le **rendement de l'actif** (`roa`, résultat net divisé par les actifs totaux), la **marge nette** (`marge`, résultat net divisé par le chiffre d'affaires) et l'**intensité de R-D** (`intensite_rd`, dépenses de R-D divisées par le chiffre d'affaires).

```
entreprises = pd.read_csv("data/entreprises.csv")
entreprises["roa"] = entreprises["resultat_net"] / entreprises["actifs_totau
entreprises["marge"] = entreprises["resultat_net"] / entreprises["chiffre_af
entreprises["intensite_rd"] = entreprises["rd"] / entreprises["chiffre_affai

e23 = entreprises[entreprises["annee"] == 2023].copy()   # une ligne par ent
```

6.2 Décrire

6.2.1 Le centre

La **moyenne** est le point d'équilibre d'une distribution ; la **médiane**, le milieu de son classement. Les deux coïncident quand la distribution est symétrique, et divergent quand elle ne l'est pas. Le chiffre d'affaires de nos 32 entreprises en 2023 en donne un exemple net.

```
ca = e23["chiffre_affaires"]
print(f"Moyenne : {ca.mean():,.0f} M$   Médiane : {ca.median():,.0f} M$   As
```

```
Moyenne : 143,776 M$   Médiane : 79,900 M$   Asymétrie : 2.01
```

La moyenne (143,776 millions de dollars) est près de deux fois la médiane (79,900 millions), parce que quelques géants de la distribution — Walmart et Amazon en

tête — tirent le point d'équilibre vers eux. Pour une variable de **taille** (ventes, actifs, effectifs, capitalisation), la médiane décrit mieux « l'entreprise typique » ; la moyenne reste utile quand c'est le **total** qui compte. Dans le doute, on rapporte les deux.

6.2.2 La dispersion

L'**écart type** mesure l'éloignement typique à la moyenne ; il est sensible aux valeurs extrêmes. L'**écart interquartile** (Q3 − Q1) délimite la moitié centrale des observations et y résiste.

```
print(e23["roa"].describe().round(3))
```

```
count    32.000
mean      0.103
std       0.099
min      -0.138
25%       0.057
50%       0.094
75%       0.144
max       0.453
Name: roa, dtype: float64
```

```
(e23.groupby("secteur")["roa"]
    .agg(n="count", moyenne="mean", mediane="median", ecart_type="std")
    .sort_values("mediane", ascending=False)
    .round(3))
```

secteur	n	moyenne	mediane	ecart_type
Technologie	7	0.186	0.170	0.146
Restauration	2	0.146	0.146	0.007
Biens de consommation	4	0.119	0.116	0.026
Médias et divertissement	1	0.111	0.111	NaN
Énergie	3	0.097	0.096	0.016
Commerce de détail	4	0.070	0.068	0.013
Pharmaceutique	4	0.076	0.046	0.096
Industrie	4	0.014	0.038	0.117
Automobile	3	0.064	0.036	0.067

La colonne `n` est la première à lire. Un secteur représenté par deux entreprises n'a pas d'écart type digne de ce nom, et une moyenne sur deux observations n'est qu'une anecdote chiffrée.

6.2.3 La forme

```
fig, (a1, a2) = plt.subplots(1, 2, figsize=(8, 3.2))
a1.hist(e23["chiffre_affaires"] / 1000, bins=12, color=CL.ink)
a1.set_title("Ventes (G$)")
a2.hist(np.log10(e23["chiffre_affaires"]), bins=12, color=CL.accent)
a2.set_title("log10(ventes en M$)")
plt.tight_layout()
plt.show()
```

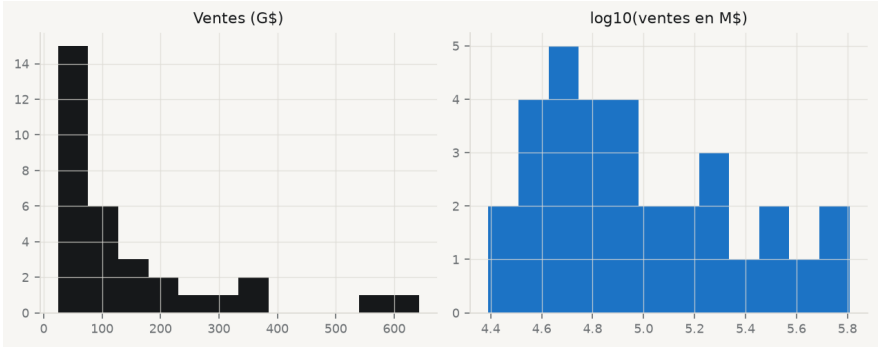


Figure 6.1. Chiffre d'affaires 2023 des 32 entreprises, en milliards de dollars US (à gauche) et en logarithme décimal (à droite). Le logarithme rend la distribution plus symétrique.

Le passage au logarithme est courant pour les variables de taille. Il a un prix : une différence de logarithmes est une différence **relative**. On ne peut plus dire « 10 milliards de plus », mais « tant de pour cent de plus ».

6.3 De l'échantillon à la population

Nos 32 entreprises ne sont pas « les entreprises ». Ce sont de grandes sociétés cotées aux États-Unis, choisies parce qu'elles sont connues — c'est-à-dire, pour la plupart, parce qu'elles ont réussi. Toute conclusion porte d'abord sur elles. L'étendre à une population plus large exige un argument, et aucun test statistique ne remplace cet argument.

Supposons tout de même que ces entreprises soient un tirage dans une population plus vaste. Un autre tirage aurait donné une autre moyenne. L'**erreur type** mesure de combien, typiquement :

$$\mathrm{ET}(\bar{x}) = \frac{s}{\sqrt{n}}$$

```
roa = e23["roa"].dropna()
n, moy, s = len(roa), roa.mean(), roa.std()
```

```
et = s / np.sqrt(n)
ic = stats.t.interval(0.95, df=n - 1, loc=moy, scale=et)
print(f"ROA moyen = {moy:.3f}, erreur type = {et:.3f}, IC 95 % = [{ic[0]:.3f}
```

```
ROA moyen = 0.103, erreur type = 0.018, IC 95 % = [0.067 ; 0.139]
```

L'intervalle de confiance à 95 % se lit ainsi : si l'on répétait l'échantillonnage un grand nombre de fois, 95 % des intervalles construits de cette façon contiendraient la vraie moyenne. Il ne dit pas qu'il y a 95 % de chances que la vraie moyenne soit dans **cet** intervalle-ci ; la vraie moyenne n'est pas aléatoire, c'est l'intervalle qui l'est. La nuance paraît scolastique ; elle devient importante dès que l'on compare deux intervalles.
Le **bootstrap** reprend la même idée sans formule : on ré-échantillonne nos propres données, avec remise, des milliers de fois, et l'on regarde comment varie la statistique.

```
rng = np.random.default_rng(60)
boot_moy = [rng.choice(roa.values, size=n).mean() for _ in range(5_000)]
boot_med = [np.median(rng.choice(roa.values, size=n)) for _ in range(5_000)]
print(f"IC bootstrap de la moyenne : [{np.percentile(boot_moy, 2.5):.3f} ; {np
print(f"IC bootstrap de la médiane : [{np.percentile(boot_med, 2.5):.3f} ; {np
```

```
IC bootstrap de la moyenne : [0.071 ; 0.137]
IC bootstrap de la médiane : [0.075 ; 0.121]
```

Les deux méthodes donnent des intervalles voisins pour la moyenne. Le bootstrap a l'avantage de s'appliquer tel quel à la médiane, pour laquelle il n'existe pas de formule simple.

6.4 Comparer

6.4.1 La logique d'un test

Un test statistique part d'une **hypothèse nulle** — « le ROA moyen est nul », « les deux groupes ont la même moyenne » — et calcule la probabilité d'observer un écart au moins aussi grand que celui des données **si cette hypothèse était vraie**. Cette probabilité est la **valeur p**.

```
t1 = stats.ttest_1samp(roa, popmean=0)
print(f"t = {t1.statistic:.2f}, p = {t1.pvalue:.2g}")
```

```
t = 5.88, p = 1.8e-06
```

Nos entreprises sont rentables en moyenne, et un ROA nul serait très incompatible avec ces données. Personne n'en doutait : l'échantillon a été construit à partir d'entreprises qui ont réussi. Le test répond rigoureusement à la question posée ; il ne dit rien de la pertinence de la question.

6.4.2 Deux groupes indépendants

Les entreprises technologiques sont-elles plus rentables que les autres ?

```
tech = e23.loc[e23["secteur"] == "Technologie", "roa"]
autres = e23.loc[e23["secteur"] != "Technologie", "roa"]
welch = stats.ttest_ind(tech, autres, equal_var=False)
print(f"Tech : n = {len(tech)}, moyenne = {tech.mean():.3f} · Autres : n = {
print(f"Écart = {tech.mean() - autres.mean():+.3f}, t = {welch.statistic:.2f
```

```
Tech : n = 7, moyenne = 0.186 · Autres : n = 25, moyenne = 0.080
Écart = +0.106, t = 1.85, p = 0.108
```

L'écart est de 11 points de ROA : considérable du point de vue d'un gestionnaire. La valeur p, 0.11, dépasse pourtant le seuil conventionnel de 5 %. Les deux constats ne se contredisent pas. Le groupe technologique compte 7 entreprises seulement, et leurs ROA sont très dispersés : Nvidia et Intel n'ont presque rien en commun en 2023. **Ces données ne suffisent pas à établir l'écart** — ce qui ne veut pas dire que l'écart n'existe pas.
Le test utilisé est celui de **Welch** (`equal_var=False`), qui n'exige pas que les deux groupes aient la même variance. Le test de Student classique l'exige ; rien ne la garantit ici. Quand les variances sont égales, les deux tests donnent presque la même réponse ; quand elles ne le sont pas, Welch protège. C'est le choix par défaut de ce cours.

6.4.3 Deux séries appariées

Tesla et Ford sont observées les mêmes années : on peut les comparer **année par année**, ce qui élimine tout ce que les deux entreprises partagent une année donnée (la conjoncture, les taux, la pandémie).

```
tf = (entreprises[entreprises["ticker"].isin(["TSLA", "F"])]
      .pivot(index="annee", columns="ticker", values="marge"))
tf["ecart"] = tf["TSLA"] - tf["F"]
tf.round(3)
```

ticker annee	F	TSLA	ecart
2010	0.051	-1.322	-1.373
2011	0.149	-1.246	-1.395
2012	0.042	-0.959	-1.001
2013	0.081	-0.037	-0.118
2014	0.009	-0.092	-0.100
2015	0.049	-0.220	-0.269

ticker annee	F	TSLA	ecart
2016	0.030	-0.096	-0.127
2017	0.049	-0.167	-0.216
2018	0.023	-0.045	-0.069
2019	0.001	-0.035	-0.036
2020	-0.010	0.023	0.033
2021	0.132	0.105	-0.027
2022	-0.013	0.155	0.167
2023	0.025	0.155	0.130
2024	0.032	0.073	0.041

```
p_tout = stats.ttest_rel(tf["TSLA"], tf["F"])
p_rec = stats.ttest_rel(tf.loc[2020:, "TSLA"], tf.loc[2020:, "F"])
print(f"2010-2024 : écart moyen {tf['ecart'].mean():+.3f}, p = {p_tout.pvalue:
print(f"2020-2024 : écart moyen {tf.loc[2020:, 'ecart'].mean():+.3f}, p = {p_re
```

```
2010-2024 : écart moyen -0.291, p = 0.048
2020-2024 : écart moyen +0.069, p = 0.121
```

Sur quinze ans, l'écart moyen est **négatif** : les pertes massives des années de démarrage de Tesla dominent la moyenne. Depuis 2020, Tesla dépasse Ford 4 années sur cinq, mais cinq années ne suffisent pas à rejeter l'égalité au seuil de 5 % ($p = 0.12$). « Tesla est plus rentable que Ford » est donc une phrase incomplète : il lui manque une période, une mesure et une incertitude.
Elle porte aussi un piège plus subtil. Si l'on choisit la période **après** avoir regardé les données — parce qu'elle donne le résultat espéré —, le test ne teste plus rien : il confirme un choix déjà fait. La période se fixe avant le calcul, et se justifie par un argument (un changement de modèle d'affaires, une première année rentable), pas par la valeur p qu'elle produit.

6.4.4 Ce qu'une valeur p n'est pas

L'American Statistical Association a jugé nécessaire, en 2016, de rappeler six principes sur la valeur p (Wasserstein et Lazar 2016). Quatre d'entre eux concernent directement le gestionnaire. Une valeur p mesure l'incompatibilité des données avec un modèle précis ; elle n'est **pas** la probabilité que l'hypothèse étudiée soit vraie ; elle ne mesure **pas** l'ampleur ni l'importance d'un effet ; et elle ne devrait **pas**, à elle seule, fonder une décision.
Le contresens le plus coûteux est de lire « non significatif » comme « aucun effet ». L'exemple du secteur technologique le montre : un écart de plusieurs points, mesuré sur sept entreprises, peut ne pas être significatif tout en étant important. Le contresens symétrique — « significatif » donc « important » — n'est pas moins

répandu, et une littérature entière montre comment la recherche de résultats significatifs, multipliée par la liberté de choisir ses spécifications, remplit les revues de résultats fragiles (Ioannidis 2005). En management, Miller et Bamberger plaident pour l'inverse : publier et lire les réplications et les **résultats nuls** comme des découvertes à part entière (C. C. Miller et Bamberger 2016).
La règle de ce cours en découle : on rapporte d'abord l'**écart en unités**, puis son **intervalle de confiance**, et seulement ensuite la valeur p.

6.5 Relier

6.5.1 Corrélations

Les entreprises qui investissent en R-D ont-elles de meilleures marges ? Le premier geste est toujours le nuage de points.

```
x = e23.dropna(subset=["intensite_rd", "marge"])
fig, ax = plt.subplots(figsize=(6.5, 4))
ax.scatter(x["intensite_rd"], x["marge"], color=CL.ink)
for _, r in x.iterrows():
    ax.annotate(r["ticker"], (r["intensite_rd"], r["marge"]), fontsize=8,
                color=CL.muted, xytext=(3, 3), textcoords="offset points")
ax.set_xlabel("Intensité de R-D (R-D / ventes)")
ax.set_ylabel("Marge nette")
plt.show()
```

```
pear = stats.pearsonr(x["intensite_rd"], x["marge"])
spear = stats.spearmanr(x["intensite_rd"], x["marge"])
print(f"n = {len(x)} · Pearson r = {pear.statistic:+.3f} (p = {pear.pvalue:.
```

```
n = 23 · Pearson r = +0.054 (p = 0.81) · Spearman rho = +0.123 (p = 0.58)
```

La corrélation de **Pearson** mesure une relation linéaire, et une seule observation extrême peut la fabriquer ou l'effacer. Celle de **Spearman** travaille sur les rangs : elle mesure une relation monotone, quelle qu'en soit la forme, et résiste aux extrêmes. Ici, sur 23 entreprises, les deux sont proches de zéro (r = +0.05, rho = +0.12) : en 2023, on ne détecte **aucune relation** entre intensité de R-D et marge nette. Neuf entreprises n'entrent pas dans ce calcul, parce qu'elles ne publient pas de ligne de R-D ; la réponse ne porte que sur celles qui le font.

```
import seaborn as sns
cols = ["roa", "marge", "intensite_rd", "chiffre_affaires", "actifs_totaux"]
fig, ax = plt.subplots(figsize=(5.6, 4.2))
sns.heatmap(e23[cols].corr(method="spearman"), annot=True, fmt=".2f",
            cmap="RdBu_r", vmin=-1, vmax=1, cbar=False, ax=ax)
plt.show()
```

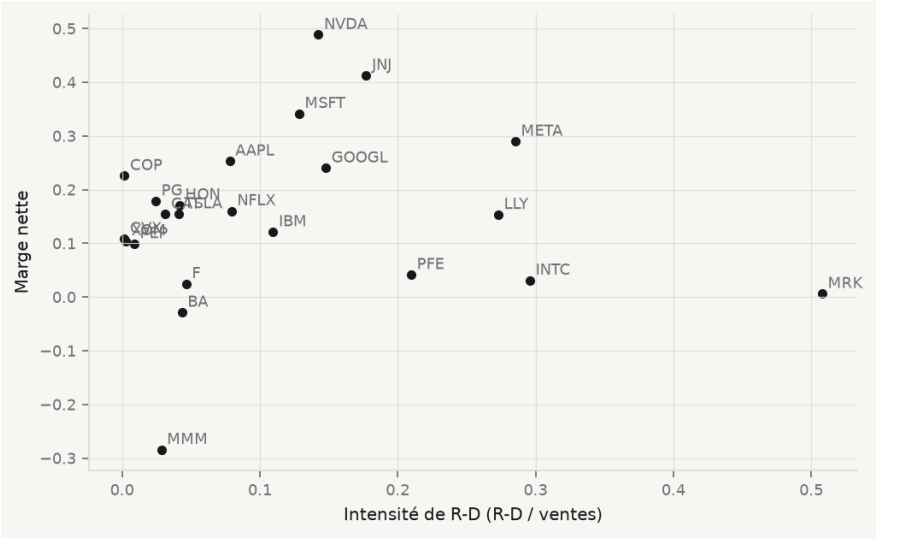


Figure 6.2. Intensité de R-D et marge nette en 2023, pour les entreprises qui publient leur R-D.

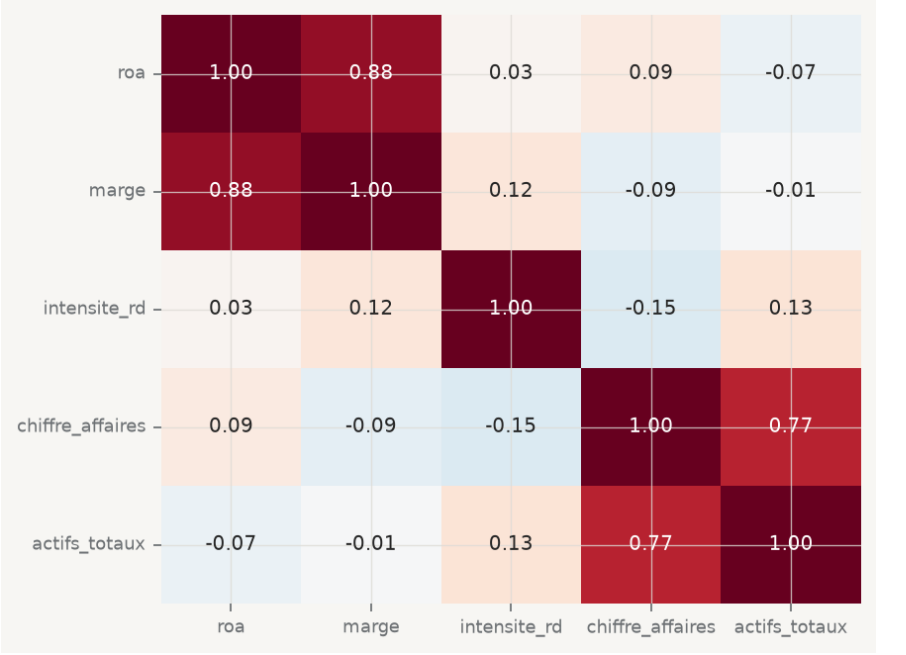


Figure 6.3. Corrélations de Spearman entre cinq variables, 2023.

6.5.2 Une première régression

La régression linéaire simple résume le nuage par une droite :

$$\text{ROA}_i = \beta_0 + \beta_1 \times \text{intensité R-D}_i + \varepsilon_i$$

```
m = smf.ols("roa ~ intensite_rd", data=e23.dropna(subset=["intensite_rd", "r
print(m.summary().tables[1])
print(f"R² = {m.rsquared:.3f}, n = {int(m.nobs)}")
```

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      0.1090      0.034      3.226      0.004      0.039      0.179
intensite_rd  -0.0288      0.198     -0.145      0.886     -0.441      0.383
==============================================================================
R² = 0.001, n = 23
```

Le coefficient de `intensite_rd` est la variation du ROA associée à une hausse d'**une unité** d'intensité de R-D — soit cent points de pourcentage ; pour un point, on le divise par cent. Ici il vaut -0.029, son intervalle de confiance contient largement zéro et le R² est quasi nul. La régression dit, avec d'autres mots, ce que disait la corrélation.

6.6 Corrélation n'est pas causalité

Même une corrélation solide n'autoriserait pas la recommandation « investissez en R-D pour être plus rentable ». Trois histoires produisent la même corrélation : la R-D crée des produits rentables (**causalité**) ; les entreprises rentables peuvent se permettre la R-D (**causalité inverse**) ; un troisième facteur, comme le **secteur**, détermine les deux (**variable confondante**). La régression ne choisit pas entre elles. C'est le dispositif de recherche qui le fait — et c'est l'objet des articles discutés aux séances 7 à 11.

6.6.1 Un panel n'est pas un échantillon plus grand

Il est tentant d'empiler les quinze années pour gagner des observations.

```
panel = entreprises.dropna(subset=["intensite_rd", "roa"])
naif = smf.ols("roa ~ intensite_rd", data=panel).fit()
grappe = smf.ols("roa ~ intensite_rd", data=panel).fit(
    cov_type="cluster", cov_kwds={"groups": panel["ticker"]})
print(f"{int(naif.nobs)} lignes, {panel['ticker'].nunique()} entreprises")
print(f"Naïf     : coef = {naif.params['intensite_rd']:+.3f}, ET = {naif.bse
print(f"Grappes  : coef = {grappe.params['intensite_rd']:+.3f}, ET = {grappe
```

```
343 lignes, 24 entreprises
Naïf     : coef = -0.167, ET = 0.041, p = 0.000
Grappes  : coef = -0.167, ET = 0.186, p = 0.369
```

Les 343 lignes ne sont pas 343 informations indépendantes : elles viennent de 24 entreprises, et Apple en 2022 ressemble beaucoup à Apple en 2023. L'erreur type naïve l'ignore et se montre trop optimiste (p = 5e-05). En **regroupant les erreurs par entreprise** (`cov_type="cluster"`), le coefficient ne change pas, mais l'erreur type est multipliée par 4.6 et la valeur p passe à 0.37. Le résultat « significatif » disparaît.

6.6.2 Contrôler le secteur

```
secteur = smf.ols("roa ~ intensite_rd + C(secteur)", data=panel).fit(
    cov_type="cluster", cov_kwds={"groups": panel["ticker"]})
print(f"Avec secteur : coef = {secteur.params['intensite_rd']:+.3f}, p = {secte
```

```
Avec secteur : coef = -0.421, p = 2.4e-09
```

`C(secteur)` ajoute une constante par secteur : on compare désormais les observations **à l'intérieur** de leur secteur. Le coefficient passe de -0.17 à -0.42, et devient nettement différent de zéro. À secteur égal, les observations (entreprise-année) à forte intensité de R-D vont avec un ROA **plus faible**. Le secteur masquait donc une relation négative.

```
fig, ax = plt.subplots(figsize=(6.5, 4))
ax.scatter(panel["intensite_rd"], panel["roa"], color=CL.muted, s=12, label="au
t = panel[panel["ticker"] == "TSLA"]
ax.scatter(t["intensite_rd"], t["roa"], color=CL.warn, s=30, label="Tesla")
for _, r in t[t["annee"] <= 2012].iterrows():
    ax.annotate(int(r["annee"]), (r["intensite_rd"], r["roa"]), fontsize=8,
                color=CL.warn, xytext=(4, 0), textcoords="offset points")
ax.set_xlabel("Intensité de R-D"); ax.set_ylabel("ROA")
ax.legend(frameon=False)
plt.show()
```

Faut-il en conclure que la R-D détruit de la rentabilité ? Non. Le graphique montre d'où vient une partie de la relation : des entreprises en phase de démarrage, comme Tesla avant 2013, dépensent beaucoup en R-D par rapport à des ventes encore faibles et perdent de l'argent. La causalité va peut-être de la **phase de croissance** à la fois vers la R-D et vers les pertes. Un coefficient n'est jamais plus solide que le nuage dont il vient.

6.7 Ce que l'on peut écrire

Le travail de ce chapitre se résume en phrases que l'on peut défendre, et en phrases que l'on ne peut pas écrire.

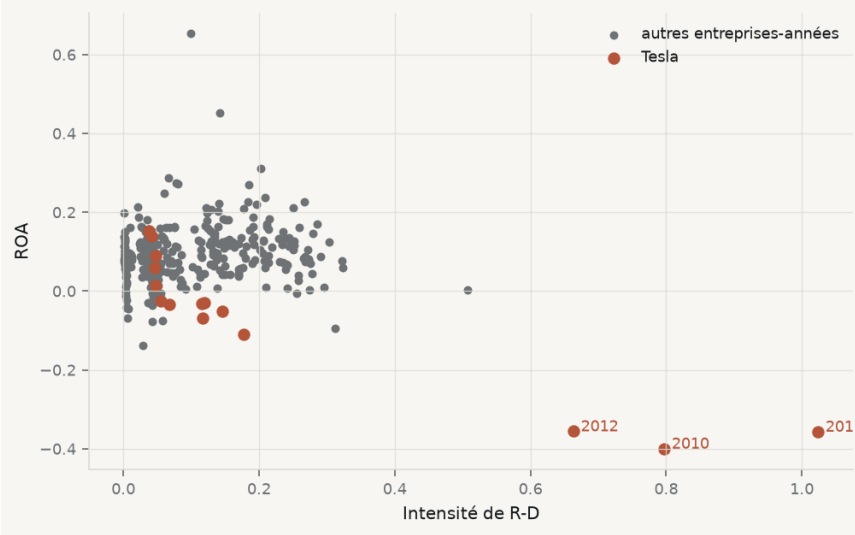


Figure 6.4. Intensité de R-D et ROA, toutes les années. Les premières années de Tesla, en rouge, se situent loin du reste du nuage.

On peut écrire	On ne peut pas écrire
Dans un échantillon de 32 grandes entreprises américaines, en 2023, on ne détecte pas de relation entre intensité de R-D et rentabilité.	La R-D ne sert à rien.
À secteur égal, les entreprises-années à forte intensité de R-D ont un ROA plus faible, sur 2010-2024.	Réduire la R-D augmenterait la rentabilité.
Depuis 2020, la marge de Tesla dépasse celle de Ford quatre années sur cinq ; l'écart n'est pas statistiquement établi.	Tesla est plus rentable que Ford.

La colonne de gauche est moins spectaculaire. C'est la seule qui survit à un lecteur attentif.

> **Pour aller plus loin.** Le travail 2 des « premiers pas » demande exactement cet exercice sur une question que vous choisissez : une source importée avec Python, des statistiques descriptives interprétées, deux graphiques, un test lu honnêtement, et un fichier de code qui reproduit chaque chiffre. Consigne : `06-statistiques-de-base/PREMIERS-PAS-TRAVAIL-2.md`.

Deuxième partie

Articles et études de cas

Chapitre 7

Des outils aux questions : les séances 7 à 13

Les six premiers chapitres ont donné les outils. Les séances 7 à 11, données par Marie-Ann et Russell, les mettent au service de questions de recherche en stratégie, à partir d'**articles** et d'**études de cas** (Harvard Business Publishing, Ivey Publishing, dépôts de recherche). Elles ne demandent pas de Python en classe ; le projet final, lui, en demande, et c'est là que les chapitres précédents servent.
Ce chapitre fait le lien : pour chaque séance, la question, la méthode, et ce que vous savez déjà faire.

Séance	Question	Méthode	Déjà vu dans ce livre
07 · Fusions-acquisitions	Les F&A créent-elles réellement de la valeur ?	Étude d'événement (MacKinlay 1997) ; performance de marché et comptable	Rendements autour d'une annonce (chapitre 4)
08 · ESG et écoblanchiment	Les mieux notés en ESG aident-ils vraiment la planète ?	Corrélation, MCO	Importer des émissions (chapitre 3) ; corrélation et MCO (chapitre 6)
09 · Groupes stratégiques	Qui sont mes véritables concurrents ?	Analyse de grappes	Écart à la moyenne du secteur (chapitre 4)
10 · Rémunération des dirigeants	Les incitatifs de long terme poussent-ils à la R-D ?	Panel : effets fixes et aléatoires ; endogénéité	Intensité de R-D ; dépendance des observations d'une même entreprise (chapitre 6)
11 · Coaching	Votre question tient-elle debout ?	Rencontres individuelles	Les premiers pas — travail 2
12–13 · Présentations	Vulgariser son sujet en cinq minutes	—	Tout

7.1 Lire un cas avec les yeux d'un analyste

Une étude de cas raconte une décision. L'habitude que les séances Python ont voulu installer s'applique à elle aussi : repérer chaque affirmation chiffrée, se demander d'où vient le chiffre, à quoi il est comparé, et ce qu'il faudrait pour le vérifier. Un cas bien écrit le permet ; un cas moins bien écrit le révèle.

7.2 Du cas au projet

Le projet de recherche du cours demande une question, un article modèle, une démarche empirique, des données, des résultats et une discussion ; Bosma et van Witteloostuijn (2024) montrent la diversité des démarches possibles en affaires internationales. L'article modèle joue le même rôle que le cas : il montre une façon de poser la question et d'y répondre. Votre travail consiste à reprendre cette démarche sur vos propres données — importées, nettoyées, représentées et testées avec les outils des chapitres 1 à 6.

Annexe A

Installation pas à pas

A.1 Python

Téléchargez **Python 3.12 ou 3.13** depuis https://www.python.org/downloads/.

- **Windows** : cochez « **Add python.exe to PATH** » dans le premier écran de l'installateur. Vérifiez ensuite dans PowerShell : `py --version`.
- **macOS** : installez le paquet `.pkg`, puis double-cliquez **`Install Certificates.command`** dans `Applications/Python 3.x`. Vérifiez dans le Terminal : `python3 --version`.

A.2 VS Code

Téléchargez **Visual Studio Code** (https://code.visualstudio.com/), puis installez les extensions **Python** et **Jupyter** (Microsoft) depuis l'onglet Extensions.

A.3 Le dossier du cours et l'environnement virtuel

Téléchargez le dossier du cours depuis https://github.com/warint/MNGT60439 (**Code → Download ZIP**), ouvrez-le dans VS Code (**Fichier → Ouvrir le dossier**), puis un terminal (**Terminal → Nouveau terminal**) :

```
# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# Windows
py -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Puis `Ctrl/Cmd + Maj + P` → **Python : Select Interpreter** → l'interpréteur de `.venv`.

A.4 Vérifier

```
python 01-installer-la-plateforme/00-pre-seance/verifier_environnement.py
```

A.5 Dépannage

Symptôme	Cause probable	Remède
`python: command not found`	Python absent du PATH	macOS : `python3` ; Windows : `py`, ou réinstaller en cochant *Add to PATH*
`ModuleNotFoundError`	Mauvais interpréteur	Activer `.venv` ; **Select Interpreter** dans VS Code
`CERTIFICATE_VERIFY_FAILED`	Certificats macOS absents	`Install Certificates.command` ; ou `export SSL_CERT_FILE=$(python -c "import certifi; print(certifi.where())")`
`FileNotFoundError: data/…`	Terminal ouvert ailleurs	`cd` vers la racine du dossier du cours
`running scripts is disabled`	Politique PowerShell	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`

Annexe B

Sources de données

Toutes les données sont **réelles** et proviennent de sources publiques. Elles sont reconstruites par `scripts/build_course_data.py` :

```
export SEC_USER_AGENT="Prénom Nom courriel@exemple.com"   # exigé par la SEC
python scripts/build_course_data.py
```

Fichier	Unité d'observation	Lignes	Source	Licence
`entreprises.csv`	entreprise × exercice (2010–2024)	480	SEC EDGAR, API XBRL *companyfacts*	Domaine public (gouvernement des États-Unis)
`entreprises_fr.csv`	idem, au format « Excel français » (`;`, virgule décimale, Windows-1252)	480	idem	idem
`pays.csv`	pays × année (2000–2023)	5 208	Banque mondiale, *World Development Indicators*, API v2	CC BY 4.0
`secteurs.csv`	secteur	9	Construit pour le cours	—
`strategie.db`	base SQLite : tables `entreprises`, `pays`, `secteurs`	—	idem	idem

Citation pour `pays.csv` : Banque mondiale, *World Development Indicators*, consulté via l'API v2 (https://data.worldbank.org), licence CC BY 4.0.

B.1 `entreprises.csv`

32 grandes entreprises cotées aux États-Unis, choisies pour couvrir neuf secteurs. Montants en **millions de dollars US**, alignés sur l'**année civile** (champ `frame` de la SEC : l'exercice qui se termine dans l'année ; les stocks sont mesurés en fin d'année).

Variable	Description
`ticker`	Symbole boursier
`entreprise`	Nom usuel
`cik`	Identifiant permanent du déclarant à la SEC
`secteur`	Secteur (voir `secteurs.csv`)
`pays_siege`	Pays du siège social (code ISO3 ; toutes `USA`)
`annee`	Année civile
`chiffre_affaires`	Revenus totaux (balises *Revenues*, *RevenueFromContractWithCustomer…*, *SalesRevenueNet*)
`resultat_net`	Résultat net (*NetIncomeLoss*)
`actifs_totaux`	Total de l'actif, fin d'année (*Assets*)
`capitaux_propres`	Capitaux propres, fin d'année (*StockholdersEquity*)
`rd`	Dépenses de recherche et développement (*ResearchAndDevelopmentExpense*)
`investissements`	Acquisitions d'immobilisations corporelles (*PaymentsToAcquirePropertyPlantAndEquipment*)

B.1.1 Pièges connus — à lire avant d'analyser

- **`rd` manque pour environ 28 % des lignes.** Les distributeurs (Walmart, Costco, Target), la restauration (McDonald's, Starbucks), Coca-Cola, Nike, General Motors et Amazon ne publient pas de ligne « R-D » sous cette balise. Amazon en dépense énormément, mais l'appelle *Technology and infrastructure*. Une valeur manquante n'est **pas** un zéro.
- **`investissements` manque pour environ 25 % des lignes**, pour des raisons semblables (balises différentes selon l'entreprise).
- Les **exercices décalés** (Apple, Microsoft, Nike, Costco…) sont rangés dans l'année civile où ils se terminent : l'exercice 2023 de Microsoft se termine le 30 juin 2023.

— Merck 2023 : la R-D (30,5 G$) inclut une acquisition de R-D en cours (Prometheus) ; c'est le chiffre publié, pas une erreur.
— Ce sont des **entreprises survivantes et géantes** : aucune conclusion ne s'étend aux PME, ni aux entreprises disparues entre 2010 et 2024.

B.2 `pays.csv`

217 économies (agrégats régionaux exclus) × 2000–2023.

Variable	Indicateur Banque mondiale	Unité
`iso3`, `pays`	—	code ISO3, nom anglais
`region`, `revenu`	Métadonnées pays	région ; groupe de revenu
`annee`	—	—
`pib_milliards_usd`	NY.GDP.MKTP.CD	milliards $ US courants
`pib_hab_usd`	NY.GDP.PCAP.CD	$ US courants par habitant
`croissance_pib`	NY.GDP.MKTP.KD.ZG	% annuel, prix constants
`population`	SP.POP.TOTL	habitants
`ide_entrants_pct_pib`	BX.KLT.DINV.WD.GD.ZS	IDE, entrées nettes, % du PIB
`commerce_pct_pib`	NE.TRD.GNFS.ZS	exportations + importations, % du PIB
`rd_pct_pib`	GB.XPD.RSDV.GD.ZS	dépense intérieure de R-D, % du PIB
`energie_renouv_pct`	EG.FEC.RNEW.ZS	part des renouvelables dans la consommation finale d'énergie

B.2.1 Pièges connus

— `rd_pct_pib` manque pour **58 %** des lignes : beaucoup de pays ne la mesurent pas, ou pas chaque année.
— `ide_entrants_pct_pib` peut être **négatif** (désinvestissement) et prend des valeurs extrêmes dans les centres financiers (îles Caïmans, Liechtenstein : plus de 1 000 % du PIB certaines années).
— `energie_renouv_pct` s'arrête en 2021 pour la plupart des pays (71 pays en 2022, aucun en 2023).
— Les dollars **courants** mélangent croissance réelle, inflation et taux de change.

B.3 Autres sources utiles pour le projet

Source	Contenu	Accès
SEC EDGAR	États financiers des sociétés cotées aux États-Unis	API `data.sec.gov` (User-Agent obligatoire)
Banque mondiale — WDI	Indicateurs pays (PIB, IDE, commerce, énergie…)	API v2, paquet `wbgapi`
Our World in Data	Émissions, énergie, santé, démographie	Fichiers CSV sur GitHub
OCDE	Statistiques des pays membres	API SDMX
FRED (Réserve fédérale de St. Louis)	Séries macroéconomiques et financières	API, clé gratuite
Statistique Canada	Tableaux de données canadiens	API Web Data Service
Yahoo Finance	Cours boursiers	Paquet `yfinance` (usage non commercial)
Harvard Dataverse, ICPSR, openICPSR	Données de réplication d'articles	Téléchargement

Annexe C

Aide-mémoire Python

Les fonctions utilisées dans les séances 1 à 6, une ligne chacune.

C.1 Lire et écrire

Code	Effet
`pd.read_csv("data/f.csv")`	Lire un CSV (aussi une URL `https://…`)
`pd.read_csv(f, sep=";", decimal=",", encoding="cp1252")`	Lire un CSV « Excel français »
`pd.read_excel("f.xlsx", sheet_name=0)`	Lire une feuille Excel
`pd.read_html(url)`	Lire les tableaux d'une page web (liste de DataFrame)
`pd.read_sql("SELECT …", con)`	Exécuter une requête SQL sur une base
`requests.get(url, params=…, headers=…).json()`	Interroger une API
`df.to_csv("f.csv", index=False)` · `df.to_parquet("f.parquet")`	Enregistrer

C.2 Regarder

Code	Effet
`df.shape` · `df.head()` · `df.info()` · `df.describe()`	Taille, premières lignes, types, résumé
`df["col"].value_counts()` · `df["col"].unique()`	Fréquences, valeurs distinctes
`df.isna().sum()`	Valeurs manquantes par colonne

C.3 Sélectionner et créer

Code	Effet
`df["col"]` · `df[["a", "b"]]`	Une colonne (Series) · plusieurs colonnes (DataFrame)
`df.loc[lignes, colonnes]` · `df.iloc[0:5, 0:3]`	Par étiquette · par position
`df[(df["a"] > 0) & (df["b"] == "x")]`	Filtrer (`&` et, `|` ou, `~` non)
`df[df["ticker"].isin(["TSLA", "F"])]`	Filtrer sur une liste
`df.sort_values("col", ascending=False)` · `df.nlargest(5, "col")`	Trier
`df["marge"] = df["resultat_net"] / df["chiffre_affaires"]`	Nouvelle colonne
`df.assign(marge=lambda d: d.resultat_net / d.chiffre_affaires)`	Nouvelle colonne, dans une chaîne

C.4 Transformer

Code	Effet
`df.groupby("secteur")["roa"].agg(["mean", "median", "count"])`	Résumer par groupe
`df.groupby(["secteur", "annee"])["roa"].transform("mean")`	Moyenne du groupe, ligne à ligne
`df.merge(autre, on="cle", how="left", validate="m:1")`	Joindre deux tables
`pd.concat([a, b])`	Empiler
`df.melt(id_vars=…, var_name=…, value_name=…)` · `df.pivot(index=…, columns=…, values=…)`	Large ↔ long
`df.groupby("ticker")["ventes"].pct_change()`	Taux de croissance par entreprise

C.5 Représenter

Code	Effet
`fig, ax = plt.subplots()` · `ax.plot(x, y)` · `fig.savefig("f.png", dpi=150)`	matplotlib

Code	Effet
`sns.histplot`, `sns.boxplot`, `sns.scatterplot(hue=…)`, `sns.relplot(col=…)`	seaborn
`px.scatter(df, x=…, y=…, size=…, hover_name=…)` · `px.choropleth(df, locations="iso3")`	plotly

C.6 Tester

Code	Effet
`df["x"].mean()`, `.median()`, `.std()`, `.quantile([.25, .75])`, `.skew()`	Descriptives
`stats.ttest_ind(a, b, equal_var=False)`	Test t de Welch
`df[["x", "y"]].corr(method="spearman")`	Corrélation
`smf.ols("y ~ x + C(secteur)", data=df).fit().summary()`	Régression MCO

Anscombe, F. J. 1973. « Graphs in Statistical Analysis ». *The American Statistician* 27 (1) : 17□21. https://doi.org/10.1080/00031305.1973.10478966.

Bosma, Bas, et Arjen van Witteloostuijn. 2024. « Machine Learning in International Business ». *Journal of International Business Studies* 55 (6) : 676□702. https://doi.org/10.1057/s41267-024-00687-6.

Cleveland, William S., et Robert McGill. 1984. « Graphical Perception : Theory, Experimentation, and Application to the Development of Graphical Methods ». *Journal of the American Statistical Association* 79 (387) : 531□54. https://doi.org/10.1080/01621459.1984.10478080.

Ioannidis, John P. A. 2005. « Why Most Published Research Findings Are False ». *PLoS Medicine* 2 (8) : e124. https://doi.org/10.1371/journal.pmed.0020124.

MacKinlay, A. Craig. 1997. « Event Studies in Economics and Finance ». *Journal of Economic Literature* 35 (1) : 13□39.

McKinney, Wes. 2010. « Data Structures for Statistical Computing in Python ». In *Proceedings of the 9th Python in Science Conference*, 56□61. https://doi.org/10.25080/Majora-92bf1922-00a.

———. 2022. *Python for Data Analysis : Data Wrangling with pandas, NumPy, and Jupyter*. 3e éd. O'Reilly. https://wesmckinney.com/book/.

Miller, C. Chet, et Peter Bamberger. 2016. « Exploring Emergent and Poorly Understood Phenomena in the Strangest of Places : The Footprint of Discovery in Replications, Meta-Analyses, and Null Findings ». *Academy of Management*

Discoveries 2 (4) : 313□19. https://doi.org/10.5465/amd.2016.0115.
Miller, Danny. 2007. « Paradigm Prison, or In Praise of Atheoretic Research ». *Strategic Organization* 5 (2) : 177□84. https://doi.org/10.1177/1476127007077558.
Wasserstein, Ronald L., et Nicole A. Lazar. 2016. « The ASA Statement on p-Values : Context, Process, and Purpose ». *The American Statistician* 70 (2) : 129□33. https://doi.org/10.1080/00031305.2016.1154108.
Wickham, Hadley. 2014. « Tidy Data ». *Journal of Statistical Software* 59 (10) : 1□23. https://doi.org/10.18637/jss.v059.i10.